

基于多无人机协同的烟幕干扰弹投放策略

摘要

本文研究无人机在来袭武器和保护目标之前投放烟幕干扰弹的策略问题。通过运动学，几何学等知识分析不同情形下的最优投放策略，建立目标优化模型，并利用差分进化等算法，保留最优个体，最大化有效遮蔽时长。

针对问题一：为求解在给定无人机有关信息情况下的有效遮蔽时长，首先，根据无人机与导弹的运动速度与方向，确定二者有关时间的坐标表达式。然后，基于干扰弹在脱离无人机时做平抛运动，爆炸后做匀速直线运动的特点，依据运动学方程进一步确定出云团中心的坐标方程。接着，利用立体几何知识证明遮蔽临界点确定准则的可行性，确定有效遮蔽的条件并分析导弹，无人机与烟幕云团的相对运动关系。最后，使用分级搜索，利用三级步长探索临界区域，提高求解结果精度，得到本问最优有效遮蔽时长为 1.406 s。

针对问题二：为实现在单无人机单干扰弹的情况下最大化有效遮蔽时长，首先，根据物体在三维空间中的运动规律，确定任意时刻的位置坐标与状态方程。接着，以无人机飞行方向、速度、引信时间与间隔时间为决策变量，最大化有效遮蔽时长为目标构建单目标优化模型，并考虑遮蔽临界点，导弹与烟幕云团相对位置等约束条件。最后，使用多阶段网格搜索，利用逐级递增的网格密度聚焦最优解附近区域，提高结果精度，得到本问最优有效遮蔽时长为 4.711 s，无人机飞行方向为 5.000°，无人机飞行速度为 132.620 m/s。

针对问题三：为实现在单无人机多干扰弹的情况下最大化有效遮蔽时长，我们先考虑无人机在投弹后飞行速度与角度可瞬时变化的一般情况，再根据题目中的约束求解当角度速度固定时最优投放策略，并将两种情况的最优解进行对比。首先，我们将本文中的有效遮蔽分为单体遮蔽与组合遮蔽，得到两种情况下的有效遮蔽时长表达式作为目标函数。在组合遮蔽的情形中，运用几何知识构建判断是否存在未遮蔽区域的步骤。接着，纳入由于多枚干扰弹而产生的新决策变量与约束条件，形成完整的多目标优化模型。最后，使用差分进化算法，计算种群中每个个体的适应度，再对个体进行变异、交叉、选择操作，不断更新最优个体，输出最佳投放策略，得到本问最优有效遮蔽时长为 6.450 s。

针对问题四：为实现在多无人机单干扰弹情况下最大化有效遮蔽时长，首先，由于不同无人机情况不同，需分别确定云团中心的坐标方程并分析单体与组合遮蔽的情况。接着，在问题三多目标优化模型基础上，调整引信时间与间隔时间等约束条件，形成多无人机多目标优化模型。最后使用改进的差分进化算法，通过分阶段约束对解空间进行限制，确保算法能够稳定收敛，获得较为稳定的最优投放策略，得到本问最优遮蔽时长为 12.200 s。

针对问题五：为实现在多无人机多干扰弹情况下最大化有效遮蔽时长，将此复杂过程拆解为多个阶段。首先，考虑导弹与无人机的高度差异，绘制无人机所在高度的水平面，与三枚导弹的运动轨迹相交并将导弹的运动轨迹投影于每一平面。接着，在第一阶段根据两个指标：无人机与导弹相对速度，无人机与交点距离来选择无人机的初始干扰对象。在第二阶段，无人机投放第一枚干扰弹，此时情况与问题四类似，采用多无人机多目标优化模型确定无人机的飞行速度与方向。在第三阶段，依据两个指标：无人机与导弹相对方向，水平垂直距离比确定无人机是否投放第二或三枚干扰弹以及干扰对象。并利用多目标优化模型实现有效遮蔽时间最大化。最后，使用分阶段求解投放策略，对解空间加以限制，对每个分问题采用改进的差分进化算法求解，得到本问最优遮蔽时长为 13.400 s。

本文特色在于，对于使用的遮蔽临界点判别准则进行了严谨的证明。同时，将复杂问题拆解为多个分过程，化繁为简，使得各子问题环环相扣，具有逻辑性。改进的差分进化算法在解空间较大情况下通过限制初始解的域能较快收敛并得到稳定的解。

关键词：运动学方程 多目标优化模型 多阶段网格搜索 差分进化算法

一、问题重述

1.1 问题背景

烟幕干扰弹常用于干扰敌方视线达到掩护目标的作用，随着技术不断发展，现已实现精准控制烟幕干扰弹定时定点投放引爆。

现对无人机投放烟幕干扰弹的策略问题展开研究，派遣在特定领域巡飞的无人机通过投放烟幕干扰弹遮蔽目标不被敌方导弹发现。本文需要建立数学模型探究烟幕干扰弹投放策略，使得真目标的有效被遮蔽时间最大。

1.2 问题重述

问题一：导弹 M1 以 300 m/s 的速度飞向假目标，无人机 FY1 以 120 m/s 的速度同样朝假目标飞行，在受领任务后可顺势调整飞行方向， 1.5 s 后投放一枚烟幕干扰弹对 M1 进行干扰，间隔 3.6 s 后起爆。通过计算求出烟幕干扰弹对 M1 的干扰时长。

问题二：仍派遣无人机 FY1 对 M1 投放一枚烟幕干扰弹进行干扰，需要计算确定 FY1 的飞行方向、飞行速度、烟幕干扰弹投放点使得对导弹的遮蔽时间尽可能长。

问题三：无人机 FY1 可分别投放三枚烟幕干扰弹对 M1 进行干扰，通过计算分别给出投放每枚烟幕干扰弹时无人机飞行方向和速度；烟幕干扰弹的投放坐标、起爆坐标和有效干扰时长。

问题四：现有 FY1、FY2、FY3 三架无人机分别投放一枚烟幕干扰弹对 M1 进行干扰。通过计算分别给出每架无人机的飞行方向和速度以及烟幕干扰弹的投放坐标、起爆坐标和有效干扰时长。

问题五：派遣全部五架无人机，每架无人机至多投放三枚烟幕干扰弹对 M1、M2、M3 三枚导弹进行干扰。通过计算给出每架无人机的飞行方向和速度以及投放全部烟幕干扰弹的坐标、起爆坐标和有效干扰时长，并给出对应被干扰的导弹编号。

二、问题分析

问题一的分析：基于无人机 FY1 与导弹 M1 的速度与运动方向，可确定二者在任意时刻的位置坐标。同时，题目给定了无人机 FY1 的引信时间与间隔时间，从而能够知道 t 时刻烟幕云团的状态（平抛运动或匀速直线下落），并根据运动学方程进一步确定出云团中心的坐标方程。接着在真目标物体上寻找多个遮蔽临界点，用几何学知识证明其合理性。然后确定有效遮蔽的定义与条件，并分析导弹，无人机与烟幕云团的相对运动关系，在给定数据条件下计算有效遮蔽时长。

问题二的分析：本问增加了无人机飞行方向、飞行速度、引信时间与间隔时间 4 个决策变量。首先针对本问单无人机单干扰弹的情况确定目标函数为有效遮蔽时间最长。接着根据物体在三维空间中的运动规律，确定任意时刻的位置坐标与状态方程。约束条件涉及云团烟幕浓度有效范围与时间，投放时间与爆炸时间差，无人机飞行角度与速度，遮蔽临界点，导弹与烟幕云团相对位置等，建立起完整的单目标优化模型。

问题三的分析：问题三在问题二基础上，增加了无人机投放烟雾干扰弹的数量。我们先考虑一般的普适情况，即无人机的飞行角度与速度在投完一次干扰弹后可瞬时调整，再根据题意固定角度速度求解最优投放策略。并将两种情况进行对比。针对本问单无人机多干扰弹的情形，我们将有效遮蔽分为单体遮蔽与组合遮蔽，分别计算这两种状况下的最长遮蔽时间。最后纳入新的约束条件与决策变量建立本文的优化模型。

问题四的分析：问题四为多无人机单干扰弹情形。多架无人机的飞行角度与速度不同，同样考虑单体遮蔽与组合遮蔽的情况，结合几何学知识分别求出两种情况的遮蔽条件并最大化有效遮蔽时长。

问题五的分析：本问为多无人机多干扰弹问题，并且每个无人机在行驶过程中的干扰对象不固定。考虑到整个过程的复杂性，可将过程拆解为几个阶段，分别确定不同阶段无人机选择干扰对象与投弹时间点，爆炸时间点的标准，使得多枚烟幕干扰弹对真目标的有效遮蔽时间最长。

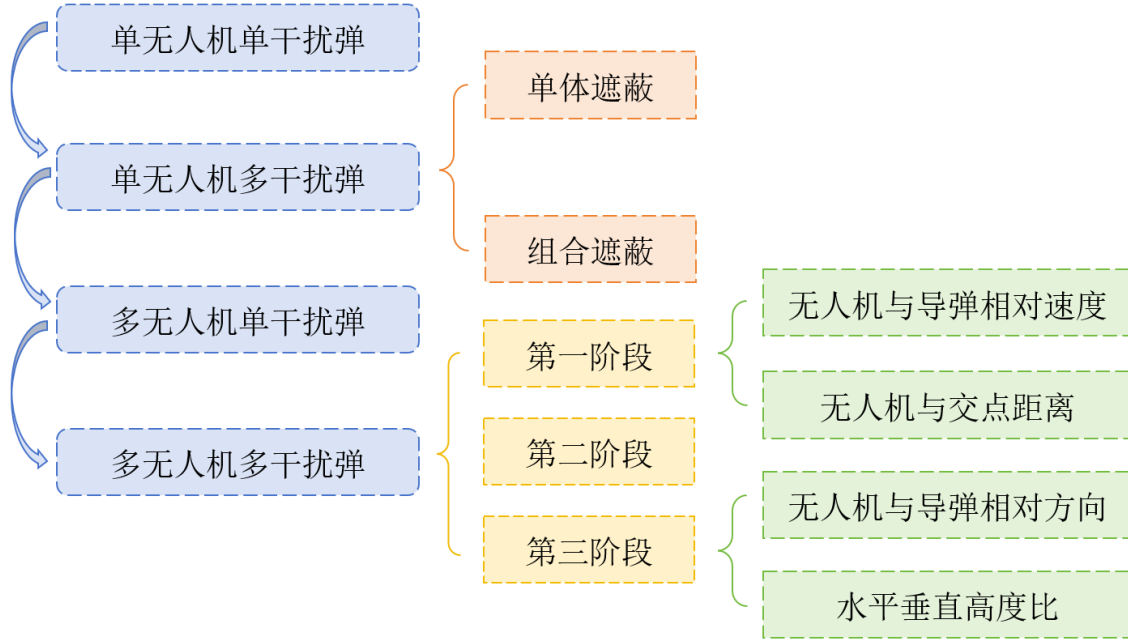


图 2-1: 全文思维导图

三、模型假设

假设1：假设本文中无人机与导弹可看作质点。

假设2：假设本文中所有空中物体不受空气阻力影响。

假设3：假设干扰弹爆炸后在 xy 平面速度分量瞬间为 0。

假设4：假设控制中心向无人机指派任务与无人机受领任务间的时间差可忽略不计。

四、符号说明

符号	含义	单位	符号	含义	单位
M_i	导弹 i	/	$x_{F_i,t}$	t 时刻无人机 i 的 x 轴坐标	m
F_i	无人机 i	/	$y_{F_i,t}$	t 时刻无人机 i 的 y 轴坐标	m
C_i	云团中心 i	/	$z_{F_i,t}$	t 时刻无人机 i 的 z 轴坐标	m
$x_{M_i,t}$	t 时刻导弹 i 的 x 轴坐标	m	θ_i	无人机 i 运动方向与 x 轴方向夹角	rad
$y_{M_i,t}$	t 时刻导弹 i 的 y 轴坐标	m	$t_0^{(i)}$	无人机 i 释放干扰弹时刻	s
$z_{M_i,t}$	t 时刻导弹 i 的 z 轴坐标	m	$t_1^{(i)}$	无人机 i 释放干扰弹爆炸时刻	s

五、问题一模型的建立与求解

5.1 坐标系的建立及坐标表示

为描述各运动物体的坐标，建立以假目标为原点，x、y、z 轴互相垂直的三维坐标系如右图所示。

初始时刻导弹 M1 与无人机 FY1 的坐标分别记为：

$$x_{M,0} = (x_M, y_M, z_M) \quad x_{F,0} = (x_F, y_F, z_F)$$

M1 距原点的距离表示为：

$$D_M = \|x_{M,0}\| = \sqrt{x_M^2 + y_M^2 + z_M^2}$$

则导弹运动的单位方向向量为：

$$d_M = -\frac{x_{M,0}}{D_M} = \left(-\frac{x_M}{D_M}, -\frac{y_M}{D_M}, -\frac{z_M}{D_M}\right)$$

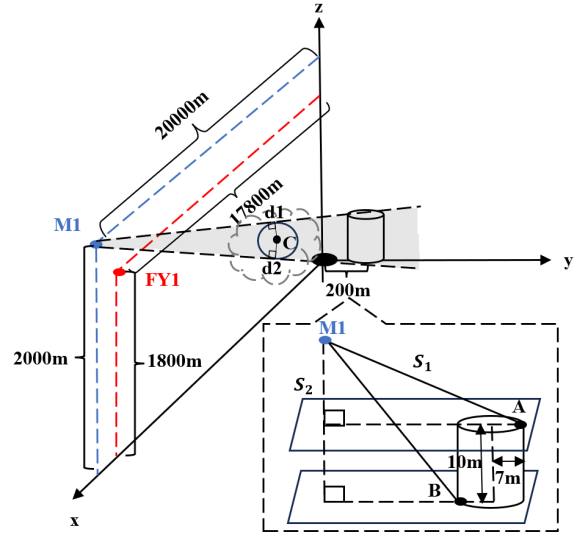


图 5-1: 无人机 FY1 与导弹 M1 坐标示意图

通过计算可得导弹以恒定速度^[4] v_M 沿 d_M 直指原点(假目标)方向运动在 t 时刻的坐标。

同理可得无人机飞行的单位方向向量为：

$$d_F = -\frac{x_{F,0}}{D_F} = \left(-\frac{x_F}{D_F}, -\frac{y_F}{D_F}, 0\right) \quad (5-1)$$

通过计算可得无人机以恒定速度 v_F 沿 d_F 方向等高度匀速直线飞行在 t 时刻的坐标。

◆ t 时刻导弹 M1 的坐标：

$$\begin{cases} x_{M,t} = x_M - \frac{x_M v_M}{\sqrt{x_M^2 + y_M^2 + z_M^2}} \cdot t \\ y_{M,t} = y_M - \frac{y_M v_M}{\sqrt{x_M^2 + y_M^2 + z_M^2}} \cdot t \\ z_{M,t} = z_M - \frac{z_M v_M}{\sqrt{x_M^2 + y_M^2 + z_M^2}} \cdot t \end{cases}$$

◆ t 时刻无人机 FY1 的坐标：

$$\begin{cases} x_{F,t} = x_F - \frac{x_F v_F}{\sqrt{x_F^2 + y_F^2}} \cdot t \\ y_{F,t} = y_F - \frac{y_F v_F}{\sqrt{x_F^2 + y_F^2}} \cdot t \\ z_{F,t} = z_F \end{cases}$$

假设无人机在 t_0 时刻扔下烟幕干扰弹， t_1 时刻烟幕干扰弹爆炸，对不同阶段运动情况进行分析：

- $t \leq t_0$ ：烟幕干扰弹尚未释放，随无人机运动（x，y，z 全部与 FY1 相同）；
- $t_0 < t \leq t_1$ ：烟幕干扰弹从无人机释放但尚未起爆，水平方向运动与无人机轨迹相同，竖直方向为初速度为 0 的自由落体运动；
- $t > t_1$ ：烟幕干扰弹起爆后形成云团，水平方向位置固定不变^[4]，竖直方向以 3 m/s 的速度匀速下降。

根据不同时段运动方式不同求得 t 时刻烟幕干扰弹爆炸中心的坐标：

$$x_{C,t} = \begin{cases} x_{F,t} & , t \leq t_1 \\ x_{F,t_1} & , t > t_1 \end{cases}, y_{C,t} = \begin{cases} y_{F,t} & , t \leq t_1 \\ y_{F,t_1} & , t > t_1 \end{cases}, z_{C,t} = \begin{cases} z_{F,t} & , t \leq t_0 \\ z_{F,t_0} - \frac{1}{2}g(t-t_0)^2 & , t_0 < t \leq t_1 \\ z_{F,t_1} - 3(t-t_1) & , t > t_1 \end{cases}$$

5.2 目标遮蔽临界点模型的建立

★ 目标遮蔽临界点的证明

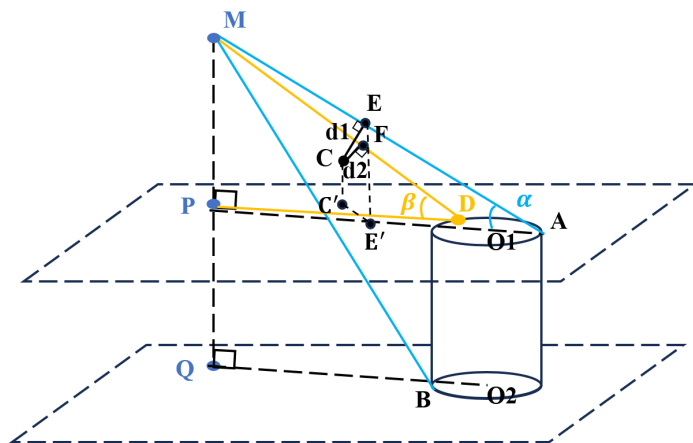


图 5-2: 图 5-1 俯视图

点 M 代表导弹，圆柱体代表真目标。过点 M 向圆柱体上顶面所在平面和下底面所在平面作垂线，交上下平面于 P、Q 两点。连接 P 与圆心 O_1 延长交上顶面圆于点 A；连接 QO_2 交下底面圆于点 B。接着证明点 A 和 B 为遮蔽临界点。

证明：任取上底面圆周上一点 D。

记 $\angle MAP = \alpha$, $\angle MDP = \beta$

作 $CE \perp MA$, $CF \perp MD$, CC', EE', FF' 垂直于上顶面。

$$\tan \alpha = \frac{MP}{PA}, \quad \tan \beta = \frac{MP}{PD}$$

$$\because PA > PD \quad \therefore \alpha < \beta$$

$$\because \alpha + \angle AEG = 90^\circ, \quad \angle AEG + \angle GEC = 90^\circ \quad \therefore \angle GEC = \alpha$$

同理 $\angle HFC = \beta$

连接 $C'E'$, $C'F'$ 可知四边形 $C'E'EC$, $C'F'FC$ 为直角梯形。

$$\text{同理 } CF = \frac{C'F'}{\sin \beta}$$

$$\because \sin \alpha > \sin \beta, \quad C'E' > C'F', \quad \therefore CE > CF$$

即云团中心 C 在圆周上任取一点与导弹 M 的连线的距离当点取到 A 点时达到最大。下表面 B 点同理。

证毕

根据题意，烟幕干扰弹爆炸后形成的云团中心 10 m 范围内的烟幕浓度在起爆 20 s 内可为目标提供有效遮蔽。为完全保护目标不被敌袭导弹发现，当且仅当 S_1 和 S_2 与爆炸中心的距离均小于 10 m 且 M1 所在点位于烟雾前方才能提供有效遮蔽作用，通过计算遮蔽临界点是否被包含在烟幕干扰弹的有效作用范围内判断真目标是否得到有效遮蔽。

为计算出真目标被完全遮蔽的情况，本文取真目标被完全遮蔽时距爆炸中心最远点作为遮蔽临界点。

已知 M1 点的坐标，将该点投影到真目标圆柱体的上下底面所在水平面，记该点在上底面 $z = 10$ 的投影点为 P，在下底面 $z = 0$ 的投影点为 Q。

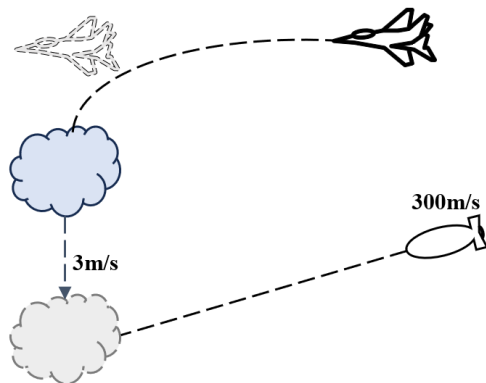


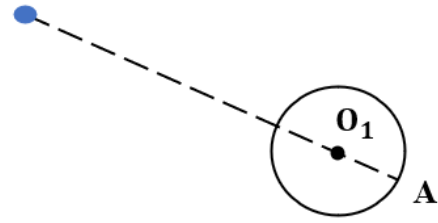
图 5-3: 烟幕干扰弹作用示意图

★ 目标被完全遮蔽的判定

■ 求解直线 S_1 与爆炸中心 C 的距离 d_1

根据图 5-1 的俯视图，投影点 P 与圆柱体上表面圆心的连线与上表面的远交点即为 A 点。记 A 点坐标为 (x_A, y_A) 。

P



在上顶面所在平面内讨论：已知圆心 O_1 坐标为 $(0, 200)$ ， P 坐标为 $(x_{M,t}, y_{M,t})$ 。

求得点 P 与圆心连线的直线解析式为：

$$y = \frac{y_{M,t} - 200}{x_{M,t}}x + 200 \quad (5-3)$$

将该解析式与圆 $O_1 : x^2 + (y - 200)^2 = 49$ 联立即可求得 A 点的坐标 (x_A, y_A) 。

图 5-4: 图 5-1 俯视图

记直线 S_1 的方向向量为 $\vec{S}_1$ ：

$$\vec{S}_1 = \overrightarrow{OM_1} - \overrightarrow{OA} = (x_{M,t} - x_A, y_{M,t} - y_A, z_{M,t} - 7) \quad (5-4)$$

$$\overrightarrow{CA} = (x_A - x_{C,t}, y_A - y_{C,t}, 7 - z_{C,t}) \quad (5-5)$$

根据向量叉积的定义可求得直线 S_1 与爆炸中心 C 的距离 d_1 为：

$$d_1 = \frac{|\vec{S}_1 \times \overrightarrow{CA}|}{|\vec{S}_1|} \quad (5-6)$$

目标被完全遮蔽需满足 $d_1 < 10$ 。

■ 求解直线 S_2 与爆炸中心 C 的距离 d_2

同理可得投影点 Q 与圆柱体下表面圆心的连线与下表面的近交点即为 B 点。记 B 点坐标为 (x_B, y_B) 。

记直线 S_2 的方向向量为 $\vec{S}_2$ ：

$$\vec{S}_2 = \overrightarrow{OM_1} - \overrightarrow{OB} = (x_{M,t} - x_B, y_{M,t} - y_B, z_{M,t} - 7) \quad (5-7)$$

$$\overrightarrow{CB} = (x_B - x_{C,t}, y_B - y_{C,t}, -z_{C,t}) \quad (5-8)$$

根据向量叉积的定义可求得直线 S_1 与爆炸中心 C 的距离 d_2 为：

$$d_2 = \frac{|\vec{S}_2 \times \overrightarrow{CB}|}{|\vec{S}_2|} \quad (5-9)$$

目标被完全遮蔽需满足 $d_2 < 10$ 。

■ M 点位于烟雾前

即线段 MA 及线段 MB 均与烟雾有交点。

烟雾形成的球体解析式为：

$$(x - x_{C,t})^2 + (y - y_{C,t})^2 + (z - z_{C,t})^2 = 100 \quad (5-10)$$

直线 MA 的解析式为：

直线 MB 的解析式为：

$$\begin{cases} x = x_A + (x_{M,t} - x_A)u \\ y = y_A + (y_{M,t} - y_A)u \\ z = 7 + (z_{M,t} - 7)u \end{cases} \quad (5-11)$$

$$\begin{cases} x = x_B + (x_{M,t} - x_B)u \\ y = y_B + (y_{M,t} - y_B)u \\ z = 7 + (z_{M,t} - 7)u \end{cases} \quad (5-12)$$

① 将烟雾球体 C 解析式与直线 MA 解析式联立得：

$$\begin{cases} (x - x_{C,t})^2 + (y - y_{C,t})^2 + (z - z_{C,t})^2 = 100 \\ x = x_A + (x_{M,t} - x_A)u \\ y = y_A + (y_{M,t} - y_A)u \\ z = z_A + (z_{M,t} - z_A)u \end{cases}$$

$$\Rightarrow [x_A + (x_{M,t} - x_A)u - x_C]^2 + [y_A + (y_{M,t} - y_A)u - y_C]^2 + [z_A + (z_{M,t} - z_A)u - z_C]^2 = 100 \quad (5-13)$$

令 $a = x_A - x_C$, $b = y_A - y_C$, $c = z_A - z_C$, $d_x = x_M - x_A$, $d_y = y_M - y_A$, $d_z = z_M - z_A$ 。
方程可化简为：

$$[a + d_x u]^2 + [b + d_y u]^2 + [c + d_z u]^2 = 100 \quad (5-14)$$

展开为 $(a^2 + b^2 + c^2) + 2u(ad_x + bd_y + cd_z) + u^2(d_x^2 + d_y^2 + d_z^2) = 100$ 。

整理为有关 u 的二次方程：

$$(d_x^2 + d_y^2 + d_z^2)u^2 + 2(ad_x + bd_y + cd_z)u + (a^2 + b^2 + c^2 - 100) = 0 \quad (5-15)$$

令 $A = d_x^2 + d_y^2 + d_z^2$, $B = 2(ad_x + bd_y + cd_z)$, $C = a^2 + b^2 + c^2 - 100$ 。

$\Delta = B^2 - 4AC > 0$ 时交点坐标为 x_{11} 和 x_{12} 。

② 同理联立烟雾球体 C 解析式与直线 MB 解析式：

$$\begin{cases} (x - x_{C,t})^2 + (y - y_{C,t})^2 + (z - z_{C,t})^2 = 100 \\ x = x_B + (x_{M,t} - x_B)u \\ y = y_B + (y_{M,t} - y_B)u \\ z = z_B + (z_{M,t} - z_B)u \end{cases}$$

得到 $(a^2 + b^2 + c^2) + 2u(ad_x + bd_y + cd_z) + u^2(d_x^2 + d_y^2 + d_z^2) = 100$ 。

其中 $a = x_B - x_C$, $b = y_B - y_C$, $c = z_B - z_C$, $d_x = x_M - x_B$, $d_y = y_M - y_B$,

$d_z = z_M - z_B$ 。

令 $A = d_x^2 + d_y^2 + d_z^2$, $B = 2(ad_x + bd_y + cd_z)$, $C = a^2 + b^2 + c^2 - 100$ 。

$\Delta = B^2 - 4AC > 0$ 时交点坐标为 x_{21} 和 x_{22} 。

目标被遮蔽需满足 $x_{M,t} > \min(x_{11}, x_{12}, x_{21}, x_{22})$ 。

综上所述，目标被完全遮蔽的条件为： d_1 、 d_2 均小于 10 且 $x_{M,t} > \min(x_{11}, x_{12}, x_{21}, x_{22})$ 。

5.3 有效遮蔽时长的表示

$$\mathbf{I}(t) = \begin{cases} 1, & d_1 \leq 10, d_2 \leq 10, & x_{M,t} > \min(x_{11}, x_{12}, x_{21}, x_{22}) \\ 0, & d_1 > 10 \text{ 或 } d_2 > 10, & x_{M,t} \leq \min(x_{11}, x_{12}, x_{21}, x_{22}) \end{cases} \quad (5-16)$$

则总有效遮蔽时长为 $T = \int_{t_1}^{t_1+20} \mathbf{I}(t) dt$ 。

5.4 模型的求解与分析

5.4.1 基于分级搜索的算法分析与设计

本问利用基于分级搜索的方法计算烟幕干扰弹的有效遮蔽时长。

本方法利用不断缩小的时间步长逐步确定有效遮蔽精准的开始时间和结束时间，从而确定烟幕干扰弹的有效遮蔽时长，具体计算步骤如下：

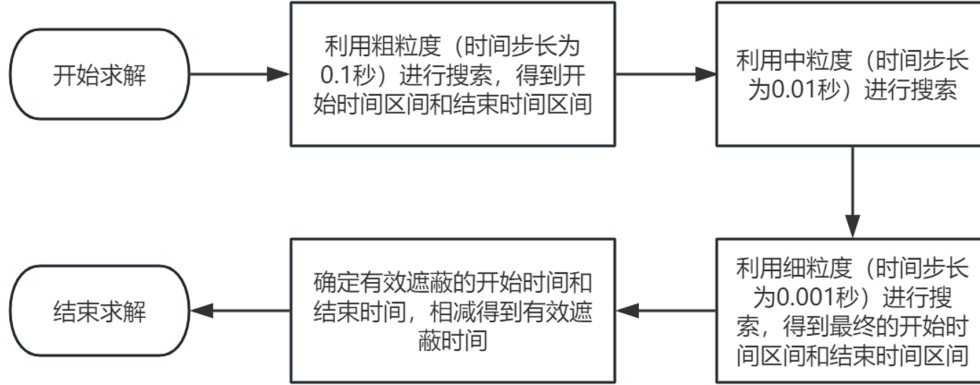


图 5-5: 问题一算法步骤流程图

Step1: 粗粒度时间搜索

在 $t = 5.1$ s 到 $t = 25.1$ s 之间每次递增 0.1 s，以 0.1 s 的步长进行计算。若某时间点 t 时烟幕干扰弹能够达到遮蔽效果，则将当前时间点的状态 $state_t$ 记为 1，若某时间点时烟幕干扰弹无法达到遮蔽效果，则将当前时间点的状态 $state_t$ 记为 0。若当前时间点的状态 $state_t$ 为 1 而前一时间点的状态 $state_{t-1}$ 为 0，则说明开始时间 $start_time$ 位于 $[t - 0.1, t]$ 区间内，将该区间加入临界区集合 $critical_zones$ ；若当前时间点的状态 $state_t$ 为 0 而前一时间点的状态 $state_{t-1}$ 为 1，则说明结束时间 end_time 位于 $[t - 0.1, t]$ 区间内，将该区间加入临界区集合 $critical_zones$ 。

Step2: 中粒度时间搜索

对步骤 (1) 中得到的开始时间区间和结束时间区间内进行步长为 0.01 s 的搜索，搜索过程与步骤 (1) 类似，最终得到区间长度为 0.01 s 的开始时间区间 $[t_start_max - 0.01, t_start_max]$ 和结束时间区间 $[t_end_min, t_end_min + 0.01]$ ，并将其加入 $critical_zones$ 。

Step3: 细粒度时间搜索

对步骤 (2) 中得到的开始时间区间和结束时间区间内进行步长为 0.001 s 的搜索，搜索过程与步骤 (1) 类似，最终得到区间长度为 0.01 s 的开始时间区间 $[t_start_max - 0.001, t_start_max]$ 和结束时间区间 $[t_end_min, t_end_min + 0.001]$ ，并将其加入 $critical_zones$ 。

Step4: 有效遮蔽的开始时间和结束时间时间的确定

对于最终得到的开始时间区间，取开始区间中 t_start_max 作为有效遮蔽的开始时间，对于结束时间区间，取结束区间中 t_end_min 作为有效遮蔽的结束时间。

Step5: 有效遮蔽时间的计算

有效遮蔽时长 $= t_end_min - t_start_max$

5.4.2 求解结果

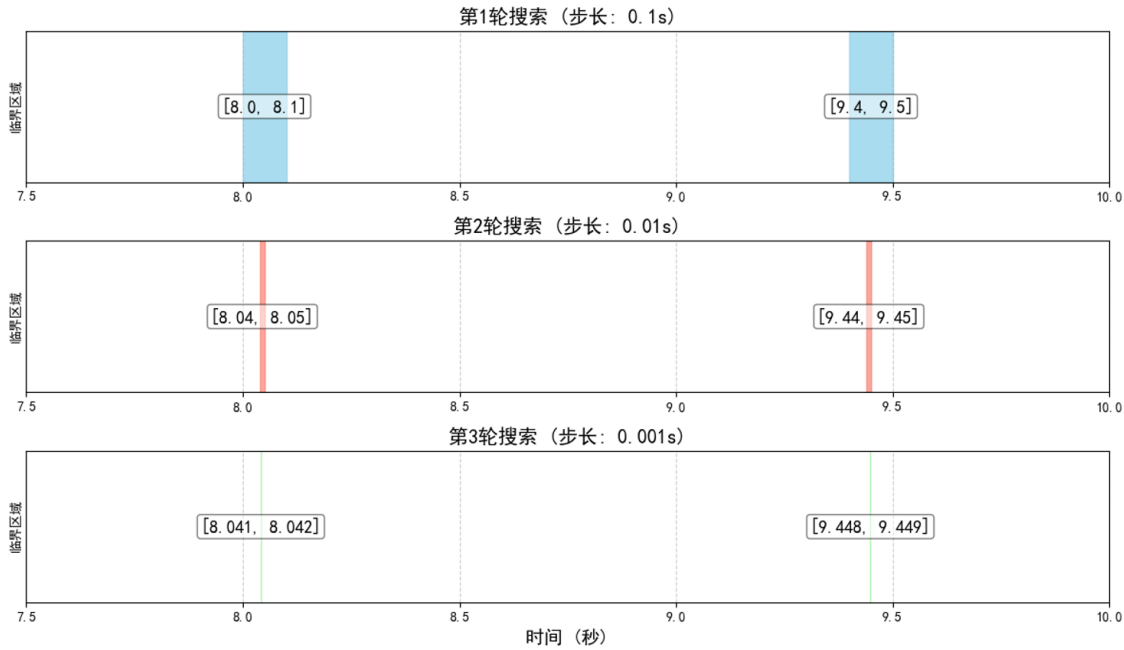


图 5-6: 逐级搜索过程

上图即为分级搜索求解有效遮蔽时长的结果：

(1) 粗粒度时间搜索

在第一轮搜索中以 0.1 秒的步长求得有效遮蔽的开始时间在区间 $[8.0, 8.1]$ 秒中，结束时间在区间 $[9.4, 9.5]$ 秒中。

(2) 中粒度时间搜索

在第二轮搜索中针对第一问求得的两个区间以 0.01 秒的步长求解，得到更为细致的开始时间及结束时间的区间。最终求得开始时间在区间 $[8.04, 8.05]$ 秒中，结束时间在区间 $[9.44, 9.45]$ 秒中。

(3) 细粒度时间搜索

在第三轮搜索中针对第二问求得的两个区间以 0.001 秒的步长求解，得到更为细致的开始时间及结束时间的区间。最终求得开始时间在区间 $[8.041, 8.042]$ 秒中，结束时间在区间 $[9.448, 9.449]$ 秒中。

(4) 有效遮蔽的开始时间和结束时间的确定

开始时间取开始时间区间的右端点，即 8.042 秒；结束时间取结束时间区间的左端点，即 9.448 秒；

(5) 有效遮蔽时间的计算

$$\text{有效遮蔽时间} = 9.448 \text{ s} - 8.042 \text{ s} = 1.406 \text{ s}$$

第一问中烟雾干扰弹在经历 1.5 s 的匀速直线运动和 3.6 s 的平抛运动后爆炸，爆炸中心的坐标为 (17188.00, 0.00, 1736.50)，而在 5.1 s 时导弹的坐标为 (18477.59, 0.00, 1847.76)。在 5.1 s 这个时刻，导弹与爆炸中心的水平距离为 $18477.59 - (17188 - 10) = 1299.59 \text{ m}$ ，而导弹的水平速度为 298.5 m/s、爆炸后的云团水平速度为。导弹将在 $1299.59 / 298.5 = 4.35 \text{ s}$ 后穿过爆炸云团，因此理论最长遮挡时长为 4.35 s，在穿过云团后导弹不可能再被遮挡。在这 4.35 s 中，云团不可能完全遮挡导弹，因此最终结果 1.406 s 的有效遮挡时间是合理的。

六、问题二模型的建立与求解

本问要求确定 FY1 的飞行方向、飞行速度、烟雾干扰弹投放点和起爆点，使得在烟雾干扰弹爆炸后的 20 s 内有效遮挡目标的时间尽可能长。

首先根据题意，判断该问题为单目标优化问题。

6.1 单目标优化模型的建立

◇ 决策变量： θ , v_F , t_0 , t_1 。

首先， θ 为无人机飞行方向与 x 轴正方向的夹角； v_F 为无人机飞行速度； t_0 为无人机投放烟雾干扰弹的时间点，用于确认烟雾干扰弹投放点的位置； t_1 为烟雾干扰弹起爆时刻，用于确认其起爆位置。

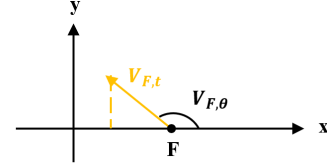


图 6-1: 无人机运动俯视图

◇ 状态方程：

t 时刻 M1 的位置 $(x_{M,t}, y_{M,t}, z_{M,t})$:

$$\begin{cases} x_{M,t} = x_M - \frac{x_M v_M}{\sqrt{x_M^2 + y_M^2 + z_M^2}} \cdot t \\ y_{M,t} = y_M - \frac{y_M v_M}{\sqrt{x_M^2 + y_M^2 + z_M^2}} \cdot t \\ z_{M,t} = z_M - \frac{z_M v_M}{\sqrt{x_M^2 + y_M^2 + z_M^2}} \cdot t \end{cases}$$

t 时刻 FY1 的位置 $(x_{F,t}, y_{F,t}, z_{F,t})$:

$$\begin{cases} x_{F,t} = x_F - v_{F,t} \cdot \cos(\pi - \theta) \\ y_{F,t} = v_{F,t} \cdot \sin(\pi - \theta) \\ z_{F,t} = z_F \end{cases}$$

设烟雾干扰弹于 t_0 时投放，此时 FY1 的位置为 $(x_{F,t_0}, y_{F,t_0}, z_{F,t_0})$ ；此后 t 时刻时 C 的位置为 $(x_{C,t}, y_{C,t}, z_{C,t})$ 。设烟雾干扰弹于 t_1 时起爆，二者的位置关系如下：

$$\textcircled{1} t_0 < t \leq t_1 \text{ 时 } \begin{cases} x_{C,t} = x_{F,t} \\ y_{C,t} = y_{F,t} \\ z_{C,t} = z_{F,t_0} - \frac{1}{2}g(t - t_0)^2 \end{cases} \quad \textcircled{2} t > t_1 \text{ 时 } \begin{cases} x_{C,t} = x_{C,t_1} = x_{F,t_1} \\ y_{C,t} = y_{C,t_1} = y_{F,t_1} \\ z_{C,t} = z_{F,t_1} - v_C(t - t_1) \end{cases}$$

直线 S1 和 S2 分别与爆炸中心 C 的距离为 $d_1 = \frac{|\vec{S}_1 \times \vec{CA}|}{|\vec{S}_1|}$, $d_2 = \frac{|\vec{S}_2 \times \vec{CB}|}{|\vec{S}_2|}$ 。

◇ 目标函数：

设 t_2 , t_3 为遮蔽的临界时刻，即当 $t \in (-\infty, t_2) \cup (t_3, +\infty)$ 时，烟幕云团无法有效遮蔽目标；而 $t \in [t_2, t_3]$ 时，烟幕云团有效遮蔽目标。优化目标是遮蔽时间最大化。

$$\max(t_3 - t_2) \quad (6-1)$$

◇ 约束条件：

① 起爆时间约束：烟幕遮蔽的开始时间大于干扰弹起爆时间：

$$t_2 - t_1 > 0 \quad (6-2)$$

② 遮蔽时间约束：因云团中心 10 m 范围内的烟幕浓度在起爆 20 s 内能有效遮蔽目标，因此遮蔽时间应在 20 s 内：

$$0 \leq t_3 - t_2 \leq 20 \quad (6-3)$$

③ 无人机高度约束：由 $z_{F,t} = \frac{1}{2}gt_{0-1}^2$ 可知 $t_{0-1} = \sqrt{\frac{2z_{F,t}}{g}}$ ，即干扰弹在 t_{0-1} 时间内落地，因此起爆与投放的时间应控制在这段时间内，即：

$$0 \leq t_1 - t_0 \leq \sqrt{\frac{2z_{F,t}}{g}} \quad (6-4)$$

④ 无人机飞行角度约束：由导弹、无人机与目标的相对位置，可规定无人机飞行方向的角度约束：

$$0 \leq \theta \leq \pi \quad (6-5)$$

⑤ 无人机速度约束：由题意得，无人机等高度匀速飞行：

$$70 \leq v_F \leq 140 \quad (6-6)$$

⑥ 临界点约束：由遮蔽的临界点时刻定义可知：

$$\begin{cases} d_1 \text{ or } d_2 > 10 & , \forall t \in (-\infty, t_2) \cup (t_3, +\infty) \\ 0 < d_1 \text{ and } d_2 \leq 10 & , \forall t \in [t_2, t_3] \end{cases} \quad (6-7)$$

⑦ 附加约束：为除去导弹穿过烟幕云团后仍满足前文规定的遮蔽条件的情况^[2]，如右图所示：M 点位于烟雾后，线段 MA 与 MB 仍均与烟雾有两交点，满足遮蔽条件。因此增加约束条件：

$$x_{M,t} > \min(x_{11}, x_{12}, x_{21}, x_{22}) \quad (6-8)$$

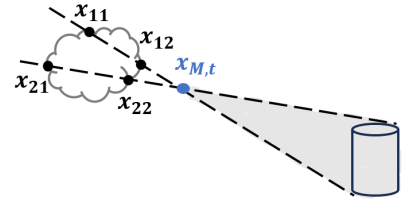


图 6-2: 特殊情况示意图

6.2 模型汇总

综上所述，本问的单目标优化模型总结如下：

$$\begin{cases} \max(t_3 - t_2) \\ d_1 \text{ or } d_2 > 10, \forall t \in (-\infty, t_2) \cup (t_3, +\infty) \\ 0 < d_1 \text{ and } d_2 \leq 10, \forall t \in [t_2, t_3] \\ t_2 - t_1 \geq 0 \\ 0 \leq t_1 - t_0 \leq \sqrt{\frac{2z_{F,t}}{g}} \\ 0 \leq \theta \leq \pi \\ 70 \leq v_F \leq 140 \\ x_{M,t} > \min(x_{11}, x_{12}, x_{21}, x_{22}) \end{cases} \quad (6-9)$$

6.3 模型的求解与分析

6.3.1 基于多阶段网格搜索的算法分析与设计

本问利用基于多阶段网格搜索的方法规划单无人机单烟幕干扰弹的投放方案。

本方法利用尺度不断缩小的由各个决策变量组成的网格，逐步确定题目条件范围内符合条件的最优的决策变量组合，从而得到烟幕干扰弹的最优投放策略，具体计算步骤如下：

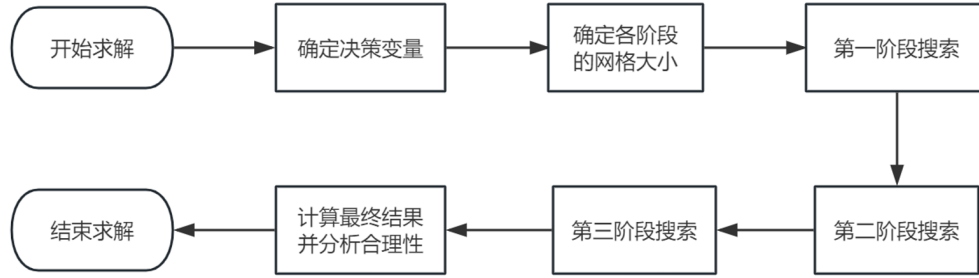


图 6-3: 问题二算法步骤流程图

Step1: 确定决策变量(θ, v_f, t_0, t_1)

- ◇ θ : 无人机 FY1 的飞行方向，范围为 $[0, 2\pi)$
- ◇ v_f : 无人机 FY1 的飞行速度，范围为 $[70, 140]$
- ◇ t_0 : 烟雾干扰弹投放的时间点
- ◇ t_1 : 烟雾干扰弹爆炸的时间点，范围为 $[0, \text{自由落体公式得到的公式}]$

Step2: 确定各阶段的网格大小

接下来每一个数对应一个决策变量的网格数量，每个维度按指定点数均匀采样：

- ▷ 第一阶段搜索: $[6, 6, 5, 5]$
- ▷ 第二阶段搜索: $[10, 6, 5, 3]$
- ▷ 第三阶段搜索: $[9, 5, 5, 4]$

Step3: 第一阶段搜索

遍历第一阶段的 $6 \times 6 \times 5 \times 5 = 900$ 种参数组合，找出最大的有效遮蔽时间时的参数，将该参数组合记录下来。

Step4: 第二阶段搜索

以第一阶段最优解为中心，收缩搜索范围到原网格间距的一半且确保不超出参数边界，生成新的搜索网格开始第二阶段搜索，遍历第二阶段的 $10 \times 6 \times 5 \times 3 = 900$ 种参数组合，找出最大的有效遮蔽时间时的参数。将该解与第一阶段搜索得到的解作比较，取最优的参数组合并记录下来。

Step5: 第三阶段搜索

以第二阶段最优解为中心，收缩搜索范围到原网格间距的一半且确保不超出参数边界，生成新的搜索网格开始第三阶段的搜索，遍历第三阶段的 $9 \times 5 \times 5 \times 4 = 900$ 种参数组合，找出最大的有效遮蔽时间时的参数。将该解与第二阶段搜索得到的解作比较，取最优的参数组合并记录下来。

Step6: 结果计算及分析

确保最优解在题目约束范围内，验证解的有效性和合理性，保存最优参数组合及该参数组合下的有效遮蔽时间。

6.3.2 求解结果

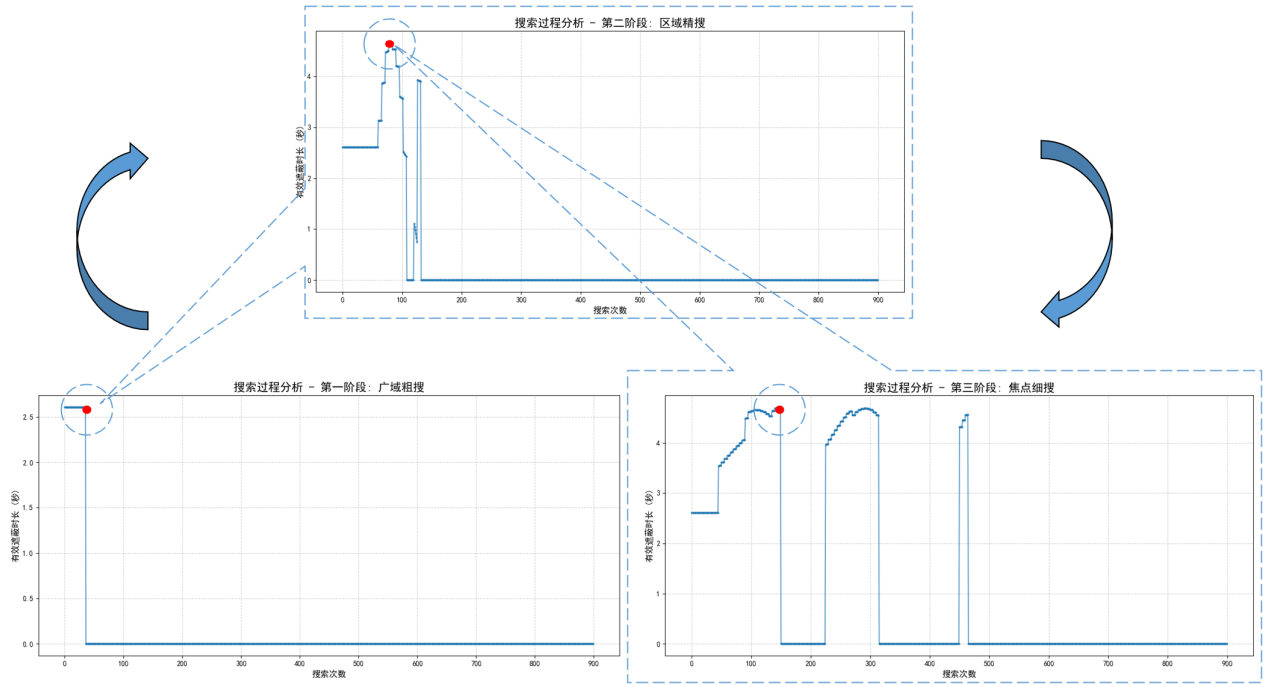


图 6-4: 搜索过程图

上图即为利用多阶段网格搜索求解单无人机单烟幕干扰弹的投放策略的求解结果:

- (1) 第一阶段搜索: 如图一所示, 在第一阶段求解过程中得到的最优解为 **2.6080 s**。
- (2) 第二阶段搜索: 如图二所示, 在第二阶段求解过程中, 通过对第一阶段的最优参数周围的小范围网格求解, 得到的最优解为 **4.6550 s**。
- (3) 第三阶段搜索: 如图三所示, 在第三阶段求解过程中, 通过对第二阶段的最优参数周围的小范围网格求解, 得到的最优解为 **4.7110 s**。对应的决策变量的值为:
 - 无人机 FY1 的飞行方向: **5.0000 °**
 - 无人机 FY1 的飞行速度: **132.6200 m/s**
 - 烟幕干扰弹投放点坐标 (X, Y, Z): **(17923.86, 10.84, 1800.00)**
 - 烟幕干扰弹起爆点坐标 (X, Y, Z): **(17923.86, 10.84, 1800.00)**

七、问题三模型的建立与求解

本问需要求解无人机 FY1 运动方向、运动速度、烟雾干扰弹 1、2、3 的投放位置和起爆位置 and 有效干扰 M1 的时长。

7.1 三次投放烟幕干扰弹时各坐标表示

已知 t 时刻导弹的位置 $(x_{M,t}, y_{M,t}, z_{M,t})$ 。根据题意, 无人机受领任务后, 可根据需要瞬时调整方向, 并以 $70 \sim 140 \text{ m/s}$ 的速度等高度匀速直线飞行, 且航向、速度一旦确定就不再调整。

我们先考虑一般情况, 即每次投完一枚干扰弹时可迅速调整方向和速度, 求解此时的最优情形。再根据题意, 考虑特殊情况, 即固定角度与速度时的最优情形。

(一) 一般情况: 每次投完一枚干扰弹时可迅速调整方向与速度

设 $t_0^{(i)}$ 为第 i 个云团被释放的时间, $t_1^{(i)}$ 为第 i 个云团爆炸时间。

释放完第 $i-1$ 枚干扰弹时, 无人机的位置为 $(x_{F,t_0^{(i-1)}}, y_{F,t_0^{(i-1)}}, z_{F,t_0^{(i-1)}})$, 此时此刻立刻根据需要调整飞行角度为 θ_i , 速度为 $v_F^{(i)}$ 。

此后, 无人机的位置为:

$$\begin{cases} x_{F,t} = x_{F,t_0^{(i)}} + v_F^{(i)} \cdot \cos \theta_i \\ y_{F,t} = y_{F,t_0^{(i)}} + v_F^{(i)} \cdot \sin \theta_i \\ z_{F,t} = z_{F,t_0^{(i)}} \end{cases}, t > t_0^{(i)} \quad (7-1)$$

云团中心 C_i 的位置为:

① $t_0^{(i)} < t \leq t_1^{(i)}$ 时:

② $t > t_1^{(i)}$ 时:

$$\begin{cases} x_{C_i,t} = x_{F,t} \\ y_{C_i,t} = y_{F,t} \\ z_{C_i,t} = z_{F,t_0^{(i)}} - \frac{1}{2}g(t - t_0^{(i)})^2 \end{cases} \quad \begin{cases} x_{C_i,t} = x_{C_i,t_1^{(i)}} \\ y_{C_i,t} = y_{C_i,t_1^{(i)}} \\ z_{C_i,t} = z_{F,t_1^{(i)}} - \frac{1}{2}g(t_1^{(i)} - t_0^{(i)})^2 - v_{c_i}(t - t_1^{(i)})^2 \end{cases} \quad (7-2)$$

为保证干扰弹在落地前爆炸, 爆炸时间与释放时间应满足 $0 \leq t_1^{(i)} - t_0^{(i)} \leq \sqrt{\frac{2z_{F,t}}{g}}$ 。

并且每架无人机投放两枚烟雾干扰弹至少 1 s, 因此 $t_0^{(i)} - t_0^{(i-1)} \geq 1$ 。

(二) 特殊情况: 固定角度与速度

此时, 令 $\theta_1 = \theta_2 = \theta_3$, $v_F^{(1)} = v_F^{(2)} = v_F^{(3)}$ 即可。

导弹与真目标上底面和下底面遮蔽临界点 A 和 B 之间连线 MA 和 MB 解析式分别为:

$$S_1: \begin{cases} x = x_A + (x_{M,t} - x_A)u \\ y = y_A + (y_{M,t} - y_A)u \\ z = z_A + (z_{M,t} - z_A)u \end{cases} \quad S_2: \begin{cases} x = x_B + (x_{M,t} - x_B)u \\ y = y_B + (y_{M,t} - y_B)u \\ z = z_B + (z_{M,t} - z_B)u \end{cases}$$

云团仅在起爆 20 s 范围内为目标提供有效遮蔽。因此云团形成的球体方程为:

$$\begin{cases} (x - x_{c_i,t})^2 + (y - y_{c_i,t})^2 + (z - z_{c_i,t})^2 = 100 & , t \leq t_1^{(i)} + 20 \\ 0 & , t > t_1^{(i)} + 20 \end{cases} \quad (i = 1, 2, 3) \quad (7-3)$$

将目标被遮蔽的情形分为单体遮蔽与组合遮蔽。

■ **单体遮蔽:** 当某一球体的球体方程与 S_1 、 S_2 有四个交点时为有效遮蔽。如图 7-1(a) 所示, C_1 为有效遮蔽, C_2, C_3 为无效遮蔽; 如图 7-1(b) 所示, C_1, C_2 为有效遮蔽, C_3 为无效遮蔽。

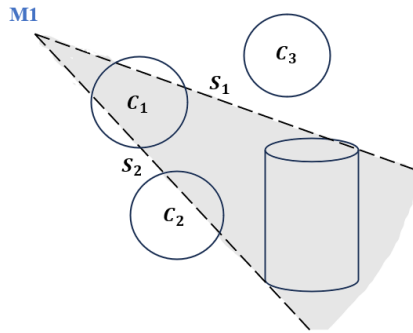


图 7-1: 单体有效遮蔽示意图

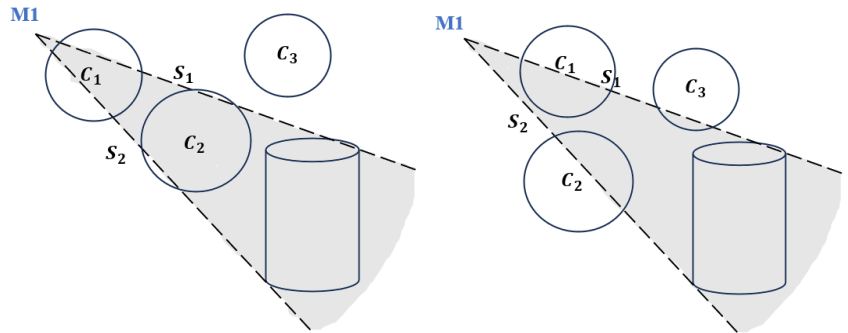


图 7-2: 组合有效遮蔽示意图

■ **组合遮蔽**:不存在单一球体完全遮蔽真目标,而是通过 2 或 3 个组合完全遮蔽真目标,参与球体的遮蔽为有效遮蔽。如图 7-2 所示, C_1, C_2 为组合有效遮蔽。

7.1.1 单体遮蔽时长

$\forall t \in (\min t_1^{(i)}, \max t_1^{(k)} + 20)$, $(i, k = 1, 2, 3)$, 联立直线解析式 S_1 与球体方程 C_i :

$$\begin{cases} (x - x_{C_i,t})^2 + (y - y_{C_i,t})^2 + (z - z_{C_i,t})^2 = 100 \\ x = x_A + (x_{M,t} - x_A)u \\ y = y_A + (y_{M,t} - y_A)u \\ z = z_A + (z_{M,t} - z_A)u \end{cases}$$

令 $a = x_A - x_{C_i}$, $b = y_A - y_{C_i}$, $c = z_A - z_{C_i}$, $d_x = x_M - x_A$, $d_y = y_M - y_A$, $d_z = z_M - z_A$ 。

方程化为 $(a^2 + b^2 + c^2) + 2u(ad_x + bd_y + cd_z) + u^2(d_x^2 + d_y^2 + d_z^2) = 100$

$\Rightarrow (d_x^2 + d_y^2 + d_z^2)u^2 + 2(ad_x + bd_y + cd_z)u + (a^2 + b^2 + c^2 - 100) = 0$

令 $A_1 = d_x^2 + d_y^2 + d_z^2$, $B_1 = 2(ad_x + bd_y + cd_z)$, $C_1 = a^2 + b^2 + c^2 - 100$ 。

某时刻存在单体遮蔽, 即 $\exists i, s.t. \Delta_1^{(i)} = B_1^2 - 4A_1C_1 > 0$ 。

同理联立直线解析式 S_2 与球体方程 C_i :

$$\begin{cases} (x - x_{C_i,t})^2 + (y - y_{C_i,t})^2 + (z - z_{C_i,t})^2 = 100 \\ x = x_B + (x_{M,t} - x_B)u \\ y = y_B + (y_{M,t} - y_B)u \\ z = z_B + (z_{M,t} - z_B)u \end{cases}$$

得 $(a^2 + b^2 + c^2) + 2u(ad_x + bd_y + cd_z) + u^2(d_x^2 + d_y^2 + d_z^2) = 100$

其中 $a = x_B - x_C$, $b = y_B - y_C$, $c = z_B - z_C$, $d_x = x_M - x_B$, $d_y = y_M - y_B$, $d_z = z_M - z_B$ 。

令 $A_2 = d_x^2 + d_y^2 + d_z^2$, $B_2 = 2(ad_x + bd_y + cd_z)$, $C_2 = a^2 + b^2 + c^2 - 100$ 。

$\exists i, s.t. \Delta_2^{(i)} = B_2^2 - 4A_2C_2 > 0$ 。

7.1.2 组合遮蔽时长

记单体遮蔽得时间集为 $\{T\}$ 。 $\forall t \in (\min t_1^{(i)}, \max t_1^{(k)} + 20) - \{T\}$, $(i, k = 1, 2, 3)$ 。

定义绘制圆锥的过程如下:

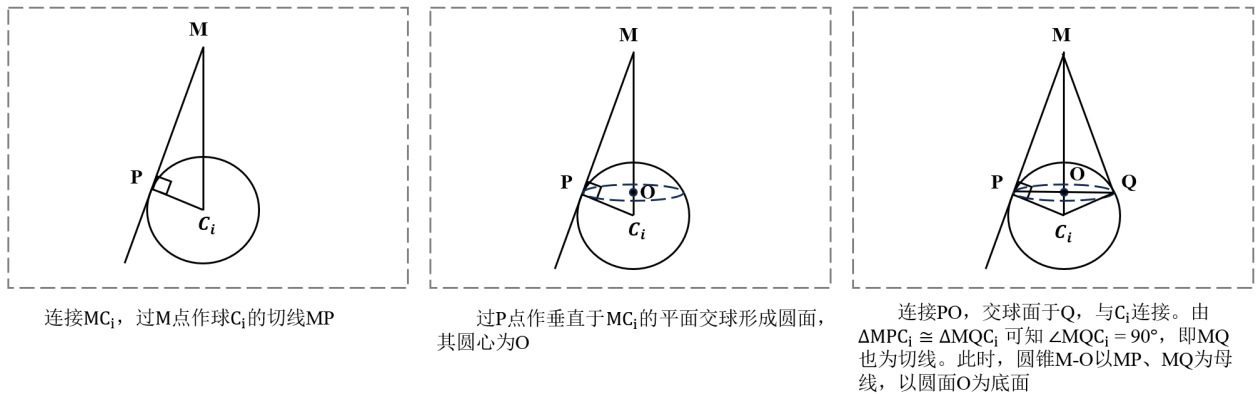


图 7-3: 圆锥绘制过程

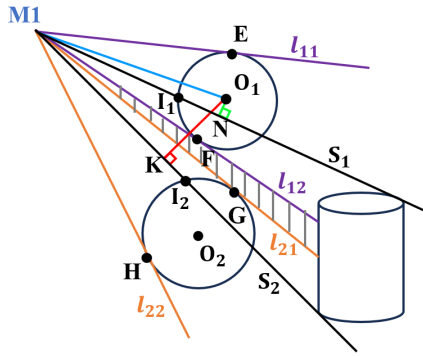


图 7-4: 组合遮蔽示意图

接下来先以两云团为例，说明未遮蔽区域。利用上述方式绘制球体 C_1 、 C_2 的圆锥。如图，此时阴影处为未遮蔽区域。

为研究是否存在未遮蔽区域，需进行以下判断：

① C_i 是否与 S_1 或 S_2 相交

是 → 有遮蔽可能性

否 → 无遮蔽效果

联立 C_i 球体方程与 S_1 或 S_2 直线方程（同单体遮蔽处联立方程） $\Delta = B^2 - 4AC > 0$

② 是否存在两个球体分别与 S_1 、 S_2 相交

是 → 有遮蔽可能性

否 → 无遮蔽效果

③ 是否存在内部未遮蔽区域，即阴影部分

是 → 无遮蔽效果

否 → 有遮蔽效果

假设 C_1 与 S_1 相交，连接 MO_1 ，作 $O_1K \perp MI_2$ 。

记 $\angle I_1MK = \theta$ ，若 $MO_1 \cdot \theta < O_1K$ ，则 O_1 在 S_1 外部；若 $MO_1 \cdot \theta > O_1K$ ，则 O_1 在 S_1 内部。

又 $\because MO_1$ 为 $\angle EMF$ 角平分线， $\therefore \angle O_1MF = \frac{1}{2} \angle EMF$

作 $O_1N \perp MI_1$ 的延长线， O_1N 的长度即为球心作 O_1 到直线 S_1 的距离 d_1 。

$$\angle O_1MN = \arcsin \frac{O_1N}{MO_1}$$

当 O_1 在 S_1 外部时， $\angle I_1MF = \angle O_1MF - \angle O_1MN$

当 O_1 在 S_1 内部时， $\angle I_1MF = \angle O_1MF + \angle O_1MN$

同理当 O_2 在 S_2 外部时， $\angle I_2MG = \angle O_2MG - \angle O_2MI_2$

当 O_2 在 S_2 内部时， $\angle I_2MG = \angle O_2MG + \angle O_2MI_2$

当有 2 个云团在同一侧时，取角度大的值作为该侧的遮蔽角度

因此，当 $\begin{cases} \Delta_1^{(i)} = \Delta_1^{(j)} > 0 \\ \Delta_2^{(i)} = \Delta_2^{(j)} \leq 0 \end{cases}$ 且 $\begin{cases} \Delta_1^{(k)} \leq 0 \\ \Delta_2^{(k)} > 0 \end{cases}$ 时：

$\begin{cases} \angle I_iMF + \angle I_kMG \geq \angle I_iMI_k, \angle I_iMF \geq \angle I_jMF \\ \angle I_jMF + \angle I_kMG \geq \angle I_jMI_k, \angle I_iMF < \angle I_jMF \end{cases}$ 表示无阴影部分，有遮蔽效果

$\begin{cases} \angle I_iMF + \angle I_kMG < \angle I_iMI_k, \angle I_iMF \geq \angle I_jMF \\ \angle I_jMF + \angle I_kMG < \angle I_jMI_k, \angle I_iMF < \angle I_jMF \end{cases}$ 表示有阴影部分，无遮蔽效果

同时，对任意一枚烟雾干扰弹，规定其不可在导弹越过或穿过后爆炸。

即对 $\forall i, x_{M,t} < \min\{x_{ij}\}, (j = 1, 2, 3, 4), x_{ij}$ 为球体方程，与 S_1, S_2 联立方程所得坐标。若 x_{ij} 不存在，令 x_{ij} 为 $+\infty$ 。

在 y 轴上，由于导弹与真目标在 y 轴上的坐标均大于 0，因此限制云团中心坐标满足： $-r < y_C < 207 + r$ ，其中 r 为云团半径。

若 t 时刻存在单体遮蔽或组合遮蔽，则 $I(t) = 1$ ；

若 t 时刻不存在单体遮蔽或组合遮蔽，则 $I(t) = 0$ ；

$$\text{则总遮蔽时间为: } \int_{\min t_1^{(i)}}^{\max t_1^{(k)}+20} \mathbf{I}(t)dt, (i, k = 1, 2, 3) \quad (7-4)$$

7.2 单无人机多干扰弹投放模型的建立

综上，本问模型为：

$$\left\{ \begin{array}{l} \int_{\min t_1^{(i)}}^{\max t_1^{(k)}+20} \mathbf{I}(t)dt, (i, k = 1, 2, 3) \\ 70 < v_F^{(1)} < 140 \\ 0 \leq t_1^{(i)} - t_0^{(i)} \leq \sqrt{\frac{2z_{F,t}}{g}}, (i = 1, 2, 3) \\ t_0^{(i)} - t_0^{(i-1)} \geq 1 \\ -r < y_C < 200 + r \\ \forall i, x_{M,t} < \min\{x_{ij}\}, (j = 1, 2, 3, 4) \end{array} \right. \quad (7-5)$$

7.3 模型的求解与分析

7.3.1 基于差分进化的算法分析与设计

本问利用基于差分进化算法规划单无人机多枚烟雾弹的投放策略。

本问题中决策变量较多，解空间较大，第二问的多阶段网格算法在本问中效率较低，不再适用。因此本问题采用差分进化算法这种启发式算法在解空间中进行更加高效的求解，求解流程如下（其中蓝色部分为该算法核心步骤）：

Step1: 根据决策变量确定解空间

在本问题中决策变量有八个，分别为：

- ① 无人机的运动方向 $[0, 2\pi]$
- ② 无人机的运动速度 $[70, 140]$
- ③ 第一枚烟雾弹被扔下的时间
- ④ 第一枚烟雾弹起爆的时间
- ⑤ 第二枚烟雾弹被扔下的时间
- ⑥ 第二枚烟雾弹起爆的时间
- ⑦ 第三枚烟雾弹被扔下的时间
- ⑧ 第三枚烟雾弹起爆的时间

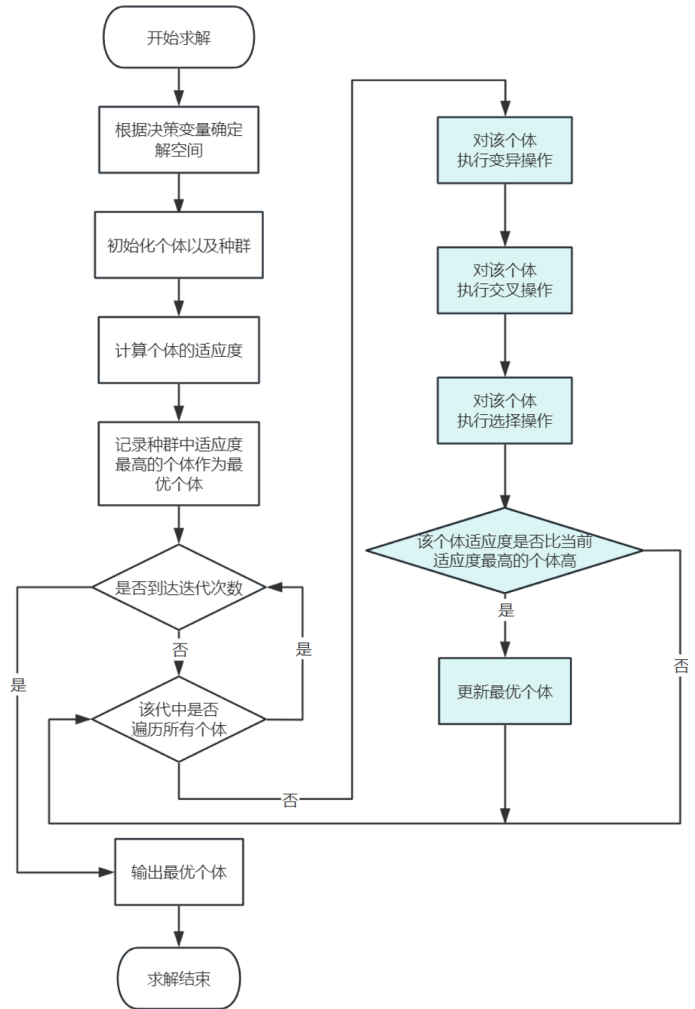


图 7-5: 问题三算法步骤流程图

Step2: 初始化个体及种群

对于差分进化算法，首先要根据步骤 1 确定的解空间对个体进行初始化。在本题中我们令种群由 100 个个体组成，每个个体代表一种可行解。在本题的背景下，种群代表所有的候选可行决策方案组成的集合，每个个体均是由步骤 1 中的八个决策变量组成的向量。如图 7-6 所示：

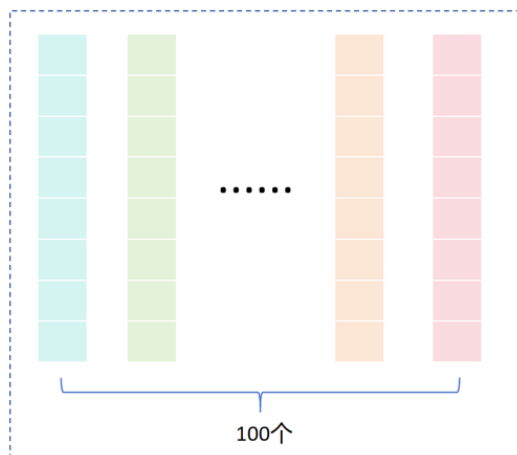


图 7-6: 初始化个体及种群示意图

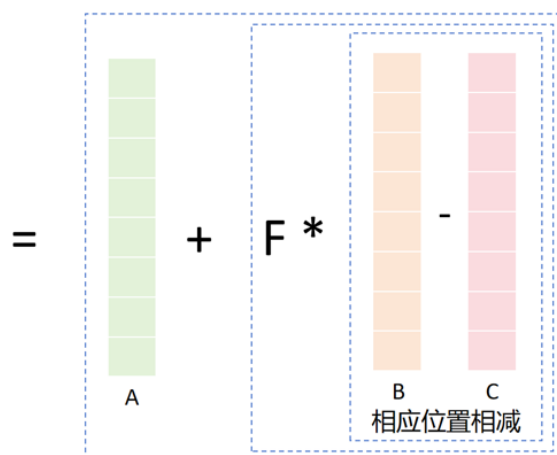


图 7-7: 个体变异操作示意图

Step3: 对初始化的种群中的每个个体计算适应度

种群中每个个体的适应度都通过问题三建立的模型的目标函数来计算，即该投放策略下真目标被完全遮蔽的时间。在对初始种群中的每个个体都进行适应度计算后，选出当前适应度最高的个体作为最优个体。本文最后需要得到种群中适应度最大的个体，即真目标被完全遮蔽的时间最长的投放策略。

Step4: 对个体进行变异操作

在种群中任意选取三个不同的个体 A、B、C，首先计算出其中两个个体，例如 B 和 C，并计算出向量差 B - C，这个差异向量代表了种群中一个随机方向和大小的探索步长。将这个向量差与变异因子 F 相乘后，加到个体 A 上，从而创建一个变异后的全新的个体 M，具体过程图 7-7 所示：

Step5: 对个体进行交叉操作

交叉操作是为了将当前目标个体与变异体结合起来，为目标体创造出一个新的个体 Q，从而增加种群的多样性。交叉操作中需要规定 CR，即交叉算子，本问题的计算中 CR 被设为 0.9。对于目标个体 N 和步骤 4 中产生的变异体 M 中的某一部分，随机在 (0,1) 中生成一个数，若这个数小于等于 CR，则新的个体中的这一部分使用变异体的值；若这个数大于 CR，则新的个体中的这一部分使用原个体的值。右图为例一次交叉操作的例子：

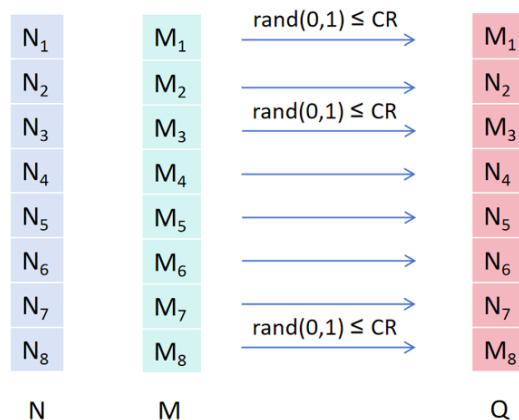


图 7-8: 交叉操作示意图

Step6: 对个体进行选择操作

计算生成的新个体 Q 的适应度，若该适应度高于原个体 N，则淘汰原个体 N，保留新生成的个体 Q。否则保留原个体 N。

7.3.2 求解结果

图 7-9 即为该问题利用差分进化算法求解的过程，目标函数为有效遮蔽时长。由图可见，随着种群的迭代，有效遮蔽时长不断上升并在 170 代时收敛，最终三枚烟雾弹对于真目标的总有效遮蔽时长为 **6.45 s**。此外，无人机的最优方向为 4.24° ，无人机的最优速度为 140 m/s 。第一枚延误干扰弹在 0 s 时扔下并在 0 s 时爆炸，爆炸的位置为 $(17800, 0, 1800)$ ，该烟雾弹最终能造成 2.65 s 的有效遮挡；第二枚延误干扰弹在 1 s 时扔下并在 1 s 时爆炸，爆炸的位置为 $(17939.62, 10.35, 1800)$ ，该烟雾弹最终能造成 4.65 s 的有效遮挡；第三枚延误干扰弹在 2 s 时扔下并在 2 s 时爆炸，爆炸的位置为 $(18079.23, 20.7, 1800)$ ，该烟雾弹最终能造成 0 s 的有效遮挡。第一枚烟雾干扰弹和第二枚烟雾干扰弹在 $4.65 + 2.65 - 6.45 = 0.85 \text{ s}$ 的时间长度内均能对真目标造成有效遮挡。无人机角度速度固定求得的烟幕干扰弹投放策略数据表已写入文件 **result1.xlsx**。

由求得到结果可以看出，3 枚烟雾弹与第二问的 1 枚烟雾弹相比有效遮蔽时间有显著的提升，但 3 枚烟雾弹起到的遮蔽作用并不均衡，2 号烟雾弹遮蔽作用最为显著，而 3 号烟雾弹并没有起到遮蔽效果。

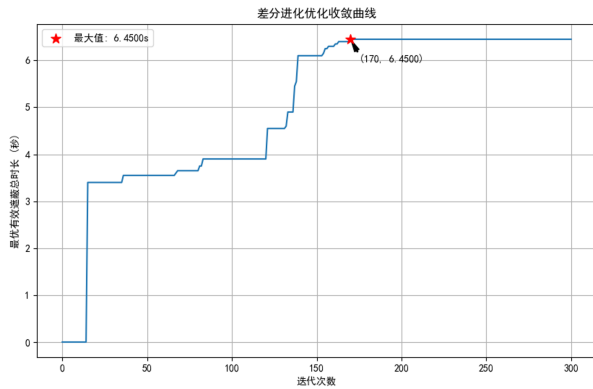


图 7-9: 无人机角度速度固定求解结果

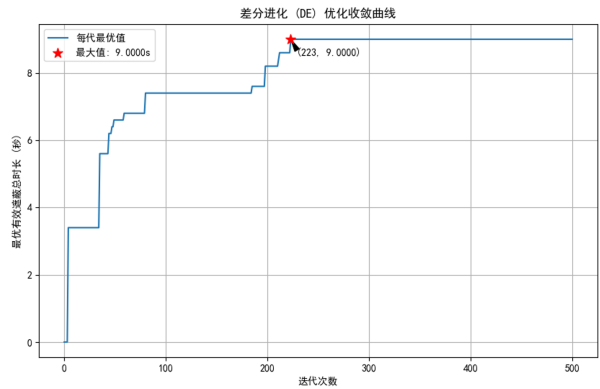


图 7-10: 无人机角度速度变化求解结果

图 7-10 即为该问题假设无人机在每次投完烟雾弹后速度和方向可以改变时利用差分进化算法求解的过程，目标函数为有效遮蔽时长。由图可见，随着种群的迭代，有效遮蔽时长不断上升并在 223 代时收敛，最终三枚烟雾弹对于真目标的总有效遮蔽时长为 **9 s**。

3 号烟幕干扰弹在这种情况下作用的提升十分明显，在角度和速度不变时，3 号烟幕干扰弹没有起到遮蔽效果，而现在它可以遮挡 2.4 s ；1 号和 2 号烟幕干扰弹的效果几乎不变，但也略微提升。总而言之，在角度和速度可以变化的情况下，3 个烟幕干扰弹都起到了显著的遮蔽作用，相较角度和速度不变的情况遮挡效果更好，且 3 个烟幕干扰弹的作用较为均衡。

表 7-1: 无人机角度速度变化求得的烟幕干扰弹投放策略数据表

无人机运动方向	无人机运动速度(m/s)	烟幕干扰弹编号	投放点x坐标(m)	投放点y坐标(m)
0	140	1	17800	0
6.09	122.27	2	17921.58	12.98
180.72	140	3	17643.37	9.5
投放点z坐标(m)	起爆点x坐标(m)	起爆点y坐标(m)	起爆点z坐标(m)	有效干扰时长(s)
1800	17800	0	1800	2.8
1800	17921.58	12.98	1800	4.6
1800	17011.15	1.6	1800	2.4

八、问题四模型的建立与求解

8.1 各坐标表示

t 时刻, 三架无人机的位置为 $(x_{F_i,t}, y_{F_i,t}, z_{F_i,t})$, $(i = 1, 2, 3)$ 。设其接收任务后瞬时调整角度为 θ_i , 速度为 v_i , $(i = 1, 2, 3)$ 。

$$\begin{cases} x_{F_i,t} = x_{F_i} + v_{F_i} \cdot \cos \theta_i \\ y_{F_i,t} = y_{F_i} + v_{F_i} \cdot \sin \theta_i \\ z_{F_i,t} = z_{F_i} \end{cases} \quad (8-1)$$

其中 $70\text{m/s} < v_{F_i} < 140\text{m/s}$ 。

设第 i 架飞机在 $t_0^{(i)}$ 时刻投放烟雾干扰弹, 在 $t_1^{(i)}$ 时刻烟雾干扰弹起爆。此时第 i 架无人机的位置为 $(x_{F_i,t_0^{(i)}}, y_{F_i,t_0^{(i)}}, z_{F_i,t_0^{(i)}})$ 。

此后时刻 t 对应的云团中心 C 的位置为 $(x_{C_i,t}, y_{C_i,t}, z_{C_i,t})$

① $t_0 < t \leq t_1$ 时:

② $t > t_1$ 时:

$$\begin{cases} x_{C_i,t} = x_{F_i,t} \\ y_{C_i,t} = y_{F_i,t} \\ z_{C_i,t} = z_{F_i,t} - \frac{1}{2}g(t - t_0)^2 \end{cases} \quad \begin{cases} x_{C_i,t} = x_{C_i,t_1^{(i)}} = x_{F_i,t_1^{(i)}} \\ y_{C_i,t} = y_{C_i,t_1^{(i)}} = y_{F_i,t_1^{(i)}} \\ z_{C_i,t} = z_{F_i,t_0^{(i)}} - \frac{1}{2}g(t_1^{(i)} - t_0^{(i)})^2 - v_{C_i}(t - t_1^{(i)}) \end{cases} \quad (8-2)$$

t 时刻云团 i 的球体方程为:

$$\begin{cases} (x - x_{c_i,t})^2 + (y - y_{c_i,t})^2 + (z - z_{c_i,t})^2 = 100 & , t \leq t_1^{(i)} + 20 \\ 0 & , t > t_1^{(i)} + 20 \end{cases} \quad (i = 1, 2, 3)$$

与问题三相同, 本问将遮蔽情形分为单体遮蔽与组合遮蔽。

若 t 时刻存在单体遮蔽或组合遮蔽, 则 $\mathbf{I}(t) = 1$;

若 t 时刻不存在单体遮蔽或组合遮蔽, 则 $\mathbf{I}(t) = 0$;

8.2 多无人机协同投放策略模型的建立

◇ 目标函数: 目标函数为真目标被遮蔽时间最长, 即 $\int_{\min t_1^{(i)}}^{\max t_1^{(k)} + 20} \mathbf{I}(t) dt$, $(i, k = 1, 2, 3)$ 。

◇ 约束条件:

① 无人机速度约束: 由题意得, 无人机等高度匀速飞行:

$$70 \leq v_{F_i} \leq 140 \quad (8-3)$$

② 无人机高度约束: 由 $z_{F_i,t} = \frac{1}{2}gt_{0-1}^2$ 可知 $t_{0-1} = \sqrt{\frac{2z_{F_i,t}}{g}}$, 即干扰弹在 t_{0-1} 时间内落地, 因此起爆与投放的时间应控制在这段时间内, 即:

$$0 \leq t_1^{(i)} - t_0^{(i)} \leq \sqrt{\frac{2z_{F_i,t}}{g}} \quad (8-4)$$

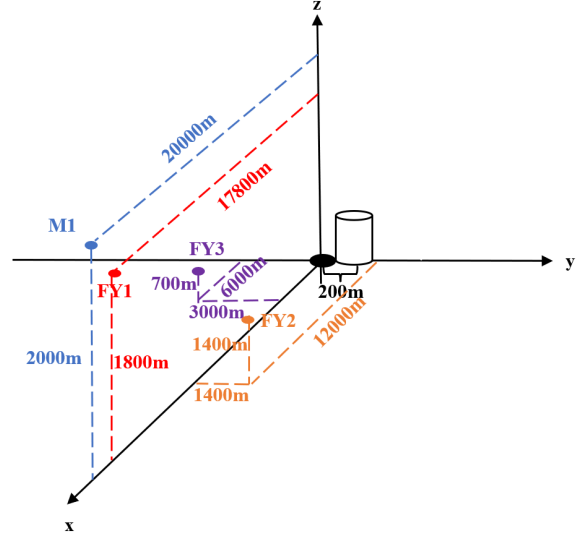


图 8-1: 问题四坐标示意图

③ 导弹与真目标相对位置约束：考虑导弹与真目标的位置在 y 轴及正半轴，因此限制云团在 y 轴的相对位置：

$$-r \leq y_{C_i} \leq 200 + r \quad (8-5)$$

④ 烟雾与导弹相对位置约束：任意一枚烟雾干扰弹不可在导弹越过或穿过后爆炸：

$$\forall i, x_{M,t} < \min\{x_{ij}\}, (j = 1, 2, 3, 4) \quad (8-6)$$

其中 x_{ij} 为球体方程，与 S_1, S_2 联立方程所得坐标。若 x_{ij} 不存在，令 x_{ij} 为 $+\infty$ 。

8.3 模型汇总

综上所述，本问的单目标优化模型总结如下：

$$\begin{cases} \int_{\min t_1^{(i)}}^{\max t_1^{(k)} + 20} \mathbf{I}(t) dt, (i, k = 1, 2, 3) \\ 70 \leq v_{F_i} \leq 140 \\ 0 \leq t_1^{(i)} - t_0^{(i)} \leq \sqrt{\frac{2z_{F_i,t}}{g}} \\ -r \leq y_{C_i} \leq 200 + r \\ \forall i, x_{M,t} < \min\{x_{ij}\}, (j = 1, 2, 3, 4) \end{cases} \quad (8-7)$$

8.4 模型的求解与分析

8.4.1 基于差分进化算法的算法分析与设计

本问利用基于差分进化算法规划多无人机单烟雾弹的投放策略。

针对本问题决策变量众多、解空间庞大的特点，我们采用差分进化算法实现对投放策略的高效求解。与常规优化问题不同的是，本问题的约束条件极为苛刻，若采用完全随机生成初始解的方式，不仅难以找到可行解，而且即使偶然获得可行解，最终优化结果也缺乏稳定性。

为此，我们设计了一种分阶段约束引导的求解策略，通过对解空间进行科学限制，确保算法能够稳定收敛到高质量解。具体求解流程如下：

Step1: 求解 FY3 的可行解

为了求解最优投放策略，要保证 FY1、FY2、FY3 这三架无人机投出的烟雾干扰弹均能造成有效遮蔽，而 FY3 投出的烟雾干扰弹能造成有效遮蔽的约束条件是三架无人机中最苛刻的，因此要首先保证 FY3 投出的烟雾干扰弹能造成有效遮蔽。

Step2: 求解 FY2 的可行解

FY2 投出的烟雾干扰弹能造成有效遮蔽的约束条件也较为苛刻，因此要在 FY3 求解成功之后求解 FY2 的可行解，保证 FY2 投出的烟雾干扰弹也能造成有效遮蔽。

Step3: 执行差分优化算法

在执行差分优化算法时，角度在中心值 $\pm 15^\circ$ 内搜索、速度在中心值 ± 10 m/s 内搜索、飞行时长在中心值 ± 8 s 内搜索、引信时长在中心值 ± 5 s 内搜索，这样可以在保证在 FY1、FY2、FY3 投出的烟雾干扰弹均能造成有效遮蔽的情况下求得有效遮蔽时间的最大值，得到较为稳定的最优投放策略。

8.4.2 求解过程及结果

求解过程如下：

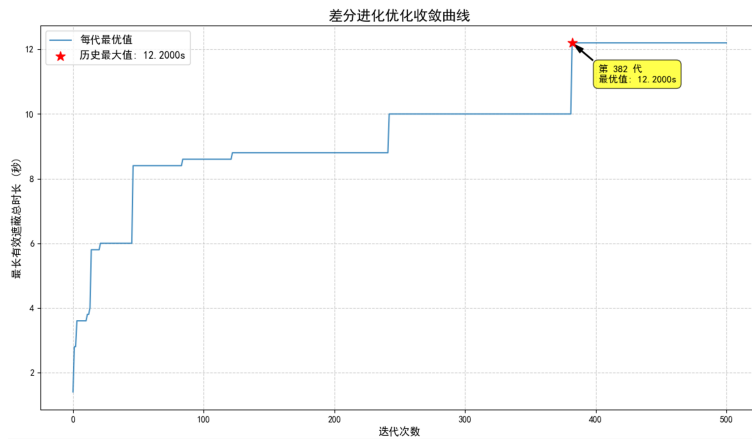


图 8-2: 收敛曲线

三枚烟雾干扰弹的爆炸点的 x 轴坐标分别为 17834.56, 11834.52, 5554.32, 任意两枚烟雾干扰弹之间的距离均大于 6000 m, 因此三枚烟雾弹的有效遮蔽时间没有交集, 有效遮蔽时长即为三枚烟雾干扰弹各自的有效遮蔽时长之和。最终求得的三架无人机烟雾弹投放策略数据表已写入文件 `result2.xlsx`。

我们对种群进行了 500 次迭代, 由上图可知适应度函数在第 382 代之后达到稳定状态, 算法收敛后得到的最长有效遮蔽时长为 12.2 s。其中 FY1 投放的烟雾干扰弹贡献了 4.6 s 有效遮蔽时长, FY2 投放的烟雾干扰弹贡献了 3.8 s 有效遮蔽时长, FY2 投放的烟雾干扰弹贡献了 3.8 s 有效遮蔽时长, 三枚烟雾弹的有效遮蔽时间没有交集且提供的遮蔽作用较为均匀。

九、问题五的模型建立与求解

根据题目中给出的真假目标和各无人机与导弹的坐标可绘制出以假目标为原点, x 、 y 、 z 轴互相垂直的三维坐标系。

通过该坐标系可看出各物体的位置关系, 为题目中的决策判断提供参考。

本问涉及 5 架无人机, 3 枚干扰弹与 3 枚导弹, 且无人机在不同时刻的干扰对象可能不同。因此决策变量较多, 过程较繁琐。为寻找到有效遮蔽时长尽可能长的投放策略, 我们将整个投弹过程拆分为几个阶段:

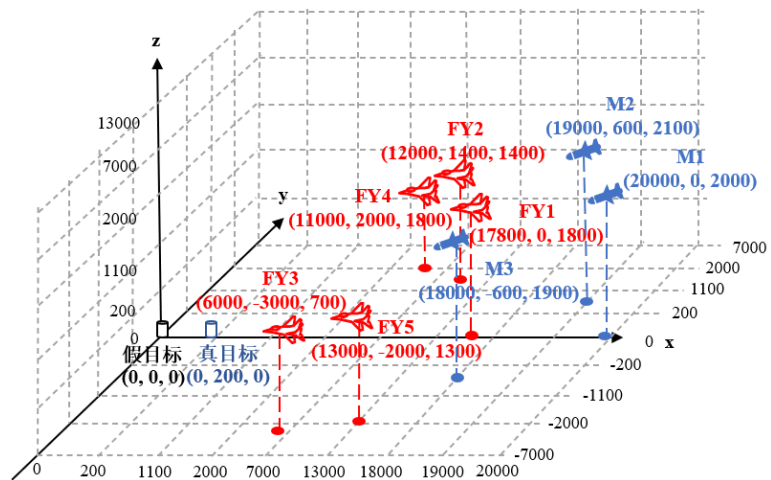


图 9-1: 问题五坐标示意图

9.1 决策阶段的建立

■ 阶段一：判断第一阶段每一无人机遮蔽的导弹

观察坐标系图可知, 所有导弹高度均高于无人机高度。由于干扰弹脱离无人机后只会下降, 这就要求无人机先靠近导弹, 再在导弹接近平面时释放干扰弹。

据此分析, 我们可先确定第一阶段某一无人机主要遮蔽哪一导弹的指标之一: 距离^[1]。

① 指标一：无人机距交点的距离

作过点 F_1 的平行于地面的平面，连接 OM_1, OM_2, OM_3 ，过该平面于点 N_1, N_2, N_3 ，连接 F_1N_1, F_1N_2, F_1N_3 ，其长度分别为 d_1, d_2, d_3 。

$$\delta_j = \begin{cases} 1 & , d_j = \min(d_1, d_2, d_3) \\ 0 & , otherwise \end{cases} \quad (9-1)$$

选择长度最短的边对应的导弹 $\sum_{j=1}^3 \delta_j M_j$ 。

② 指标二：无人机先于导弹到达交点 N

为了最大化有效遮蔽时长，排除通过指标一选定的导弹通过无人机对应的平面时，无人机还未到达交点 N_j 。

假设导弹与无人机同时到达节点的极限情况，此时无人机释放干扰弹。

因为干扰弹在 xy 平面内的速度小于导弹在 xy 平面内的速度，当干扰弹在 z 轴速度 $v_{C,z}$ 小于导弹在 z 轴的速度 $v_{M,z}$ 时，二者距离变远，因此考虑当 $v_{C,z} = v_{M,z}$ 时的情况。

干扰弹在 z 轴方向的速度为 $v_{C,z} = gt$ ，导弹沿 z 轴的速度为 $v_{M,z} = v_M \frac{z_M}{\sqrt{x_M^2 + y_M^2 + z_M^2}}$ 。

当 $v_{C,z} = v_{M,z}$ 时， $t = \frac{v_M \cdot z_M}{g\sqrt{x_M^2 + y_M^2 + z_M^2}}$

干扰弹在 xy 平面的速度为 v_F ，在 t 时刻，前进距离为：

$$l_C = \frac{v_F \cdot v_M \cdot z_M}{g\sqrt{x_M^2 + y_M^2 + z_M^2}} \quad (9-2)$$

导弹在 xy 平面的速度为 $v_{M,xy} = v_M \frac{\sqrt{x_M^2 + y_M^2}}{\sqrt{x_M^2 + y_M^2 + z_M^2}}$ ，在 t 时刻，前进距离为：

$$l_M = \frac{v_M^2 \cdot z_M \cdot \sqrt{x_M^2 + y_M^2}}{g\sqrt{x_M^2 + y_M^2 + z_M^2}} \quad (9-3)$$

$$\frac{l_M}{l_C} = \frac{v_M^2 \cdot z_M \cdot \sqrt{x_M^2 + y_M^2}}{g\sqrt{x_M^2 + y_M^2 + z_M^2}} \cdot \frac{g\sqrt{x_M^2 + y_M^2 + z_M^2}}{v_F \cdot v_M \cdot z_M} = \frac{v_M \sqrt{x_M^2 + y_M^2}}{v_F \sqrt{x_M^2 + y_M^2 + z_M^2}} \quad (9-4)$$

根据题中所给的导弹与无人机位置，计算得 $\min \frac{l_M}{l_C} \approx 2.13$ 。即导弹在 xy 平面前进距离远大于干扰弹在 xy 平面前进距离。

此后，若干扰弹仍不爆炸，虽然其在 z 轴方向速度大于导弹，但其 xy 平面方向速度仍小于导弹，因此 xy 平面距离差距持续扩大，远大于云团可影响距离。

因此，确认指标二：无人机先于导弹到达交点 N。

无人机到达交点时间 $t_F = \frac{d_j}{v_F}$ ，导弹到达交点时间 $t_M = \frac{\sqrt{(z_M - z_F)^2 + x_M^2 + y_M^2}}{v_M}$ 。

$$t_F < t_M, \text{ 即 } \frac{v_M}{v_F} < \frac{\sqrt{(z_M - z_F)^2 + x_M^2 + y_M^2}}{d_j}。$$

■ 阶段二：无人机投放第 1 枚干扰弹

此时，每架无人机有确定的对应干扰的导弹，且先只考虑它投掷于第 1 枚干扰弹时的状况，与问题四情况类似，即多架无人机各投放 1 枚干扰弹对某一导弹进行干扰。因此沿用问题四的模型：

t 时刻, 无人机的位置为 $(x_{F_i,t}, y_{F_i,t}, z_{F_i,t})$, $(i = 1, 2, 3, 4, 5)$ 。
 设其接收任务后, 瞬时调整角度为 θ_i , 速度为 v_i , $(i = 1, 2, 3, 4, 5)$ 。
 通过无人机协同策略投放模型

$$\begin{cases} \int_{\min t_1^{(i)}}^{\max t_1^{(k)}+20} \mathbf{I}(t)dt, (i, k = 1, 2, 3) \\ 70 \leq v_{F_i} \leq 140 \\ 0 \leq t_1^{(i)} - t_0^{(i)} \leq \sqrt{\frac{2z_{F_i,t}}{g}} \\ -r \leq y_{C_i} \leq 200 + r \\ \forall i, x_{M,t} < \min\{x_{ij}\}, (j = 1, 2, 3, 4) \end{cases}$$

确认无人机飞行方向、飞行速度、第一枚烟幕干扰弹投放点与起爆点。

■ 阶段三: 无人机投放第 2、3 枚干扰弹

在投掷完第一枚干扰弹后, θ_i, v_i 速度已确定, 此时不再考虑固定单一导弹, 而是重新确定下一干扰对象 (可能是多个)。

设无人机 F_i 在 $t_0^{(i)}$ 时刻释放第一枚干扰弹, 此时其位置为 $(x_{F_i,t_0^{(i)}}, y_{F_i,t_0^{(i)}}, z_{F_i,t_0^{(i)}})$ 。

$$\begin{cases} x_{F_i,t_0^{(i)}} = x_{F_i} + v_i \cos \theta_i t_0^{(i)} \\ y_{F_i,t_0^{(i)}} = y_{F_i} + v_i \sin \theta_i t_0^{(i)} \\ z_{F_i,t_0^{(i)}} = z_{F_i} \end{cases} \quad (9-5)$$

三枚导弹位置为 $(x_{M_j,t_0^{(i)}}, y_{M_j,t_0^{(i)}}, z_{M_j,t_0^{(i)}})$ 。

$$\begin{cases} x_{M_j,t_0^{(i)}} = x_{M_j} - v_{M_j} \frac{x_M}{\sqrt{x_{M_j}^2 + y_{M_j}^2 + z_{M_j}^2}} t_0^{(i)} \\ y_{M_j,t_0^{(i)}} = y_{M_j} - v_{M_j} \frac{y_M}{\sqrt{x_{M_j}^2 + y_{M_j}^2 + z_{M_j}^2}} t_0^{(i)} \\ z_{M_j,t_0^{(i)}} = z_{M_j} - v_{M_j} \frac{z_M}{\sqrt{x_{M_j}^2 + y_{M_j}^2 + z_{M_j}^2}} t_0^{(i)} \end{cases} \quad (9-6)$$

① 指标一: 相对方向^[3]

将三枚导弹 M_j 的运动轨迹与 $t_0^{(i)}$ 时刻 M_j 空间所在点投影至无人机 F_i 所在的平面。
 连接 M_j' 与 F_i , $M_j'F_i$ 长度记为 d_j 。作垂直于 $M_j'F_i$ 的线段 R_iH_i 。

M_j' 的坐标为 $(x_{M_j',t_0^{(i)}}, y_{M_j',t_0^{(i)}}, z_{M_j',t_0^{(i)}})$ 。

$$\begin{cases} x_{M_j',t_0^{(i)}} = x_{M_j} - v_{M_j} \frac{x_M}{\sqrt{x_{M_j}^2 + y_{M_j}^2 + z_{M_j}^2}} t_0^{(i)} \\ y_{M_j',t_0^{(i)}} = y_{M_j} - v_{M_j} \frac{y_M}{\sqrt{x_{M_j}^2 + y_{M_j}^2 + z_{M_j}^2}} t_0^{(i)} \\ z_{M_j',t_0^{(i)}} = z_{F_i,t_0^{(i)}} \end{cases} \quad (9-7)$$

无人机 F_i 位置指向导弹 M_j' 的方向向量为:

$$\overrightarrow{F_i M_j'} = (x_{F_i, t_0^{(i)}} - x_{M_j', t_0^{(i)}}, y_{F_i, t_0^{(i)}} - y_{M_j', t_0^{(i)}}, z_{F_i, t_0^{(i)}} - z_{M_j', t_0^{(i)}}) \quad (9-8)$$

若无人机 F_i 的方向向量 $\vec{v}$ 与向量 $\overrightarrow{F_i M_j'}$ 夹角为锐角，如下图所示，则有可能产生遮蔽效果。

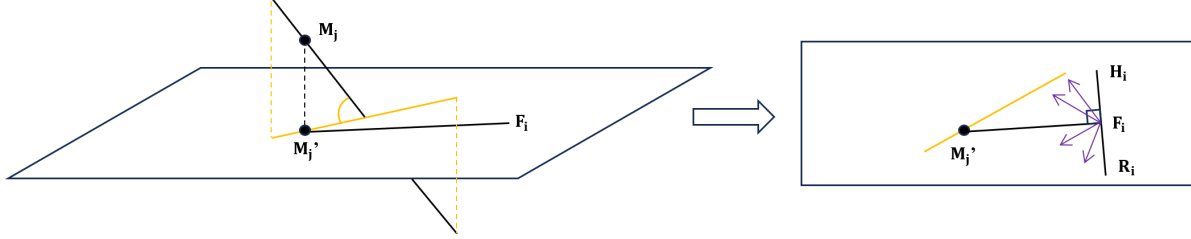


图 9-2: 方向向量夹角示意图

$$\text{即 } w_j = \begin{cases} 1 & , \frac{\vec{v} \cdot \overrightarrow{F_i M_j'}}{\|\vec{v}\| \cdot \|\overrightarrow{F_i M_j'}\|} > 0 \\ 0 & , \frac{\vec{v} \cdot \overrightarrow{F_i M_j'}}{\|\vec{v}\| \cdot \|\overrightarrow{F_i M_j'}\|} \leq 0 \end{cases} \quad (9-9)$$

② 指标二：水平距离垂直高度比

在考虑无人机与导弹的相对方向后，还需考虑二者在水平面与竖直方向的距离比。若导弹在水平面上距离远且无人机距导弹投影点距离较近，则无人机投弹后，时间引信时序对爆炸事件的选择范围更大，增加有效遮蔽时长发生的概率。

因此，选择水平距离比垂直高度比值最小的导弹作为下一干扰目标。

即：当 $z_{M_j} > z_{F_i}$ 时， $h_{ij} = z_{M_j} - z_{F_i}$ 为导弹所在高度与无人机所在高度的高度差。

$\exists j, s.t. h_{ij} > 0$ 时，令 $a = \max\{\frac{h_{ij}}{d_j}\}, (j = 1, 2, 3)$ 。

$$w_j' = \begin{cases} 1 & , \frac{h_{ij}}{d_j} = a \\ 0 & , \frac{h_{ij}}{d_j} < a \end{cases}, (j = 1, 2, 3) \quad (9-10)$$

若所有导弹均在水平面下方，此时利用空间距离来进行选择。

即： $\forall j, h_{ij} < 0$ 时，令 $b = \min \|M_j F_i\|, (j = 1, 2, 3)$ 。

$$w_j' = \begin{cases} 1 & , \|M_j F_i\| = b \\ 0 & , \|M_j F_i\| > b \end{cases}, (j = 1, 2, 3) \quad (9-11)$$

选择符合两个条件（相对方向和距离比）的导弹作为第二干扰对象。即：

$$M^{(2)} = \sum_{j=1}^3 w_j \cdot w_j' \cdot M_j \quad (9-12)$$

接着继续利用多无人机协同策略投放模型确认无人机飞行方向、飞行速度、第一枚烟雾干扰弹投放点与起爆点。第二次投放后到第三次选择干扰对象过程与上述做法相同。

9.2 模型的求解与分析

9.2.1 基于改进的差分算法的算法分析与设计

本问利用基于改进的差分算法规划多无人机多烟雾弹的投放策略。

针对本问题决策变量众多、解空间及其庞大的特点，在算法方面我们采用与第四问相似的差分进化算法实现对投放策略的高效求解。本问题的约束条件比问题四更加苛刻，若采用完全随机生成初始解的方式，几乎不可能找到可行解，且运行时间极长，十分低效。

为此，我们设计了一种分阶段求解投放策略，加之对解空间的限制，确保算法能够稳定收敛到高质量解。具体求解流程如下：

Step1: 将无人机和导弹根据位置关系分组

本题中有五架无人机和三个导弹，根据无人机与导弹之间的距离将其初始分为三组，通过降低解空间的大小，极大的降低了算法的复杂度。

Step2: 第一阶段求解（求解投放第一个烟雾干扰弹的最优策略）

第一阶段的求解即五架无人机投放第一个烟雾干扰弹时的决策变量求解，此时决策变量有四个：无人机的飞行速度、无人机的飞行角度、投放烟雾干扰弹的时间以及烟雾干扰弹爆炸的时间。通过步骤一的分组，该问题化简为了三个独立的“问题四”，即多无人机单干扰弹的策略求解问题。

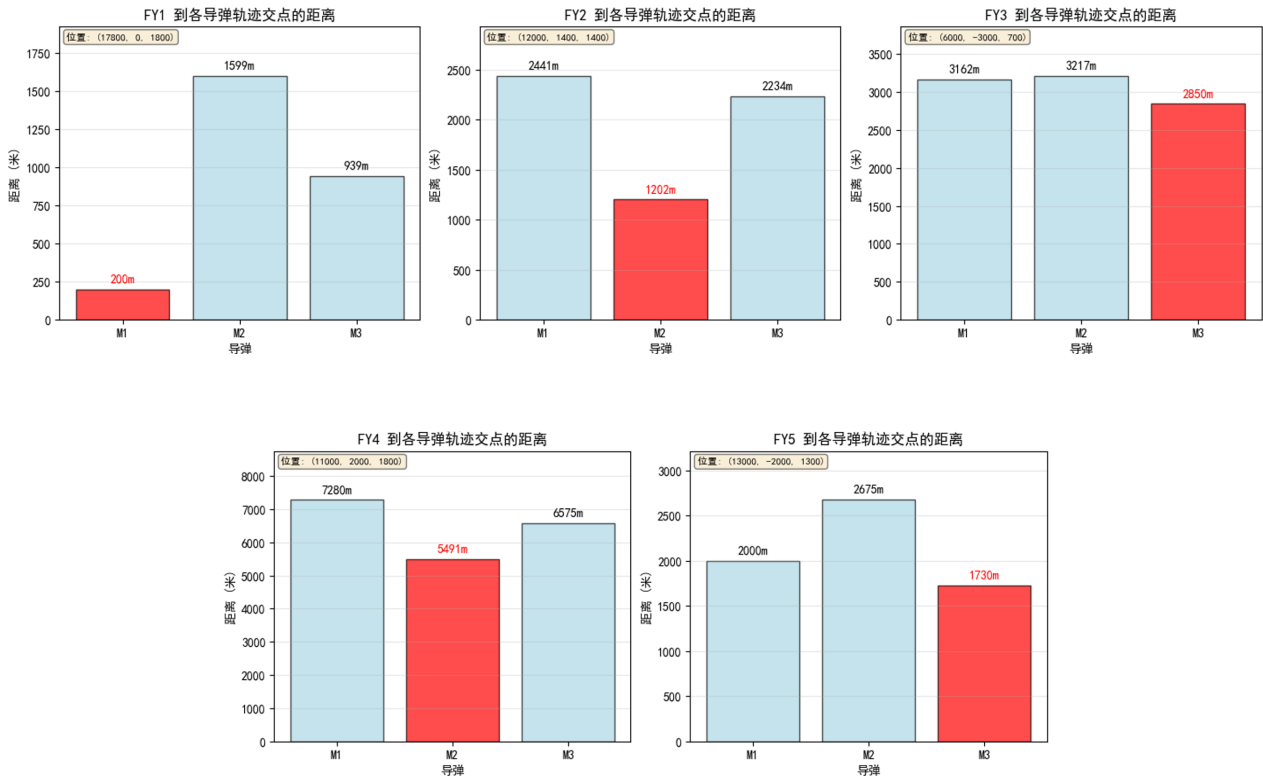


图 9-3: 第一阶段求解过程图

上图即为第一阶段的求解的分组过程。由图可见，无人机 FY1 到导弹 M1 的距离最近、无人机 FY2 到导弹 M2 的距离最近、无人机 FY3 到导弹 M3 的距离最近、无人机 FY4 到导弹 M2 的距离最近、无人机 FY5 到导弹 M3 的距离最近，因此分为以下三组：

- 1) (FY1,M1) 2) (FY2,FY4,M2) 3) (FY3,FY5,M3)

对这三组分别求解即可得到每架无人机投放第一个烟雾干扰弹的最优策略。

Step3: 第二阶段求解（求解投放后续烟雾干扰弹的最优策略）

由于无人机在起飞后不能改变其飞行方向和飞行速度，在步骤二的求解中每架无人机的飞行方向和飞行速度已经确定，因此在后续的求解过程中决策变量仅有两个，即投放烟雾干扰弹的时间以及烟雾干扰弹爆炸的时间。利用本问模型中建立的几何关系即可求解是否投放烟雾弹，以及这枚烟雾弹可以对哪枚导弹起到遮蔽作用。

Step4: 结果分析

将第一阶段得到的结果与第二阶段得到的结果求并集即可得到最终的最优遮蔽时长以及最优投放策略。

9.2.2 求解结果

最终求得的求解结果为 13.4 s。其中无人机 FY1 投放了三枚烟雾干扰弹,贡献了 4.4 s 的有效遮蔽时长，无人机 FY2 投放了三枚烟雾干扰弹，贡献了 3.8 s 的有效遮蔽时长，无人机 FY3 投放了两枚烟雾干扰弹，贡献了 2.6 s 的有效遮蔽时长，无人机 FY4 投放了三枚烟雾干扰弹，贡献了 3.6 s 的有效遮蔽时长，无人机 FY5 投放了两枚烟雾干扰弹，贡献了 3.4 s 的有效遮蔽时长。从结果可以看出，五架无人机的遮蔽作用较为均匀，其中 FY2 的遮蔽效果最为显著。虽然五架无人机的有效遮蔽时间区间有少量交集，但五架无人机配合较为默契，利用求解得到的最优策略达到了对于真目标的有效遮蔽。

部分求解结果如图所示：

无人机编号	无人机运动方向	无人机运动速度 (m/s)	烟雾干扰弹编号	烟雾干扰弹投放 点的x坐标 (m)	烟雾干扰弹投放 点的y坐标 (m)
FY1	161.58	132.5	1	17787.35	4.21

烟雾干扰弹投放 点的z坐标 (m)	烟雾干扰弹起爆 点的x坐标 (m)	烟雾干扰弹起爆 点的y坐标 (m)	烟雾干扰弹起爆 点的z坐标 (m)	有效干扰时长 (s)	干扰的导弹编号
1800	17777.05	7.64	1799.97	1.8	M1

图 9-4: 部分求解结果

十、模型的评价与推广

10.1 模型的评价

10.1.1 模型的优点

1. 全文建模始终基于数学物理理性思维，利用几何学，运动学等理论知识严谨分析物体状态与相关关系。
2. 在模型建立过程中，对运用的遮蔽临界点判别准则给出了严谨的数学证明。
3. 通过限制初始解的解空间这一策略，改进了传统的启发式算法，使得最终求解结果更加稳定。

10.1.2 模型的缺点

1. 本文中的模型忽视了现实存在的误差，如空气阻力影响，而这种影响在现实中难以消除。

2. 在多无人机多干扰弹的情形中，为了简化模型，我们将整个过程分为不同阶段讨论，可能忽视了某些极限情况。

10.2 模型的推广

1. 本文模型可推广至其他领域，如应用于气象追踪，追踪与探测极端天气。无人机或飞机追踪台风眼、龙卷风等移动的气象现象，在特定的位置投下“探空仪”，收集内部气象数据。

2. 随着全局优化问题复杂度的增加以及对求解精度要求的不断提高，差分进化算法的基也在持续扩展。例如，自适应差分进化（SaDE）、复合差分进化（CoDE）、以及 L-SHADE 等算法的变体和改进策略被相继提出，以应对高维、多模态、计算复杂度高的复杂优化挑战。

参考文献

- [1]丁佳麟,陈春生,李庆伟,等.烟幕干扰效果评估指标及计算方法[J].红外,2024,45(03):29-39.
- [2]田浩,荣凯,赵书敏.烟幕遮蔽雷达型精确制导武器的作战效果研究[J].航空兵器,2023,30(06):75-80.
- [3]肖云,曹丹,李成龙,等.基于高空无人机平台的多模态跟踪数据集[J].中国图象图形学报,2025,30(02):361-374.
- [4]DeepSeek, DeepSeek-R1-0528, 深度求索(DeepSeek), 2025-09-04

附录

附录A：问题计算结果

10.3 问题三投放策略计算结果

无人机运动方向	无人机运动速度 (m/s)	烟幕干扰弹编号	烟幕干扰弹投放 点的x坐标 (m)	烟幕干扰弹投放 点的y坐标 (m)	烟幕干扰弹投放 点的z坐标 (m)	烟幕干扰弹起爆 点的x坐标 (m)	烟幕干扰弹起爆 点的y坐标 (m)	烟幕干扰弹起爆 点的z坐标 (m)	有效干扰时长 (s)
4.24	140	1	17800	0	1800	17800	0	1800	2.65
4.24	140	2	17939.62	10.35	1800	17939.62	10.35	1800	4.65
4.24	140	3	18079.23	20.7	1800	18079.23	20.7	1800	0
注：以x轴为正向， 逆时针方向为正，取 值0~360（度）。									

10.4 问题四投放策略计算结果

无人机编号	无人机运动方向	无人机运动速度 (m/s)	烟幕干扰弹投放 点的x坐标 (m)	烟幕干扰弹投放 点的y坐标 (m)	烟幕干扰弹投放 点的z坐标 (m)	烟幕干扰弹起爆 点的x坐标 (m)	烟幕干扰弹起爆 点的y坐标 (m)	烟幕干扰弹起爆 点的z坐标 (m)	有效干扰时长 (s)
FY1	5.78	80.79	17834.56	3.5	1800	17881.68	8.27	1798.32	4.6
FY2	259.45	76.37	11834.52	511.73	1400	11743.13	21.18	1190.82	3.8
FY3	100.17	114.13	5554.32	-515.42	700	5443.2	103.87	551.24	3.8
注：以x轴为正向， 逆时针方向为正，取 值0~360（度）。									

10.5 问题五投放策略计算结果

无人机编号	无人机运动方向	无人机运动速度 (m/s)	烟幕干扰弹编号	烟幕干扰弹投放 点的x坐标 (m)	烟幕干扰弹投放 点的y坐标 (m)
FY1	161.58	132.5	1	17787.35	4.21
FY1	161.58	132.5	2	17428.77	123.63
FY1	161.58	132.5	3	17234.13	188.46
FY2	210.32	138.96	1	11137.38	895.52
FY2	210.32	138.96	2	10928.39	773.3
FY2	210.32	138.96	3	10845.66	724.92
FY3	112.2	140	1	4903.63	-313.44
FY3	112.2	140	2	4826.1	-123.44
FY3			3		
FY4	259.36	136.27	1	10685.15	324.07
FY4	259.36	136.27	2	10627.7	18.29
FY4	259.36	136.27	3	10597.45	-142.75
FY5	145.28	131.2	1	11092.63	-678.29
FY5	145.28	131.2	2	10785.38	-465.38
FY5			3		
注：以x轴为正向， 逆时针方向为正， 取值0~360（度） 。					

烟幕干扰弹投放 点的z坐标 (m)	烟幕干扰弹起爆 点的x坐标 (m)	烟幕干扰弹起爆 点的y坐标 (m)	烟幕干扰弹起爆 点的z坐标 (m)	有效干扰时长 (s)	干扰的导弹编号
1800	17777.05	7.64	1799.97	1.8	M1
1800	17316.33	161.08	1796.08	1.4	M1
1800	16972.14	275.71	1778.72	1.2	M2
1400	10921.17	769.08	1384.08	0.8	M2
1400	10694.22	636.35	1381.33	1.6	M2
1400	10765.74	678.18	1397.83	2	M2
700	4884.21	-265.85	699.34	1.2	M3
700	4776.13	-1	695.63	2	M1
1800	10635.69	60.83	1781.07	1.8	M2
1800	10610.41	-73.74	1797.69	2.2	M1
1800	10562.24	-330.18	1790.4	1	M3
1300	10646.91	-369.43	1216.29	2.2	M3
1300	10411.66	-206.41	1241.15	1.6	M3
					M1

附录B：问题代码

问题一代码

```
import math
import numpy as np
import sympy as sp
import time

G = 9.8
rs = 100.0
xm, ym, zm = 20000, 0, 2000
xfy, yfy, zfy = 17800, 0, 1800
vm, vfy = 300, 120
# 求解A1, A2两点
x, y = sp.symbols('x y')
circle_eq = sp.Eq(x**2 + (y - 200)**2, 49)
line_eq = sp.Eq(y, -0.01 * x + 200)
solutions = sp.solve((circle_eq, line_eq), (x, y))
sol1, sol2 = solutions[0], solutions[1]
x1, y1 = float(sol1[0]), float(sol1[1])
x2, y2 = float(sol2[0]), float(sol2[1])
if x1 < x2:
    xs, ys = x1, y1
    xb, yb = x2, y2
else:
    xs, ys = x2, y2
    xb, yb = x1, y1
s_p = np.array([xs, ys, 7.0])
bp = np.array([xb, yb, 0.0])
pm1 = np.array([float(xm), float(ym), float(zm)])
NORM_M = np.linalg.norm(pm1)
pfy = np.array([float(xfy), float(yfy), float(zfy)])
NORM_FY = np.linalg.norm(pfy[:2])
tc = 5.1
xct = xfy - xfy / NORM_FY * vfy * tc
yct = yfy - yfy / NORM_FY * vfy * tc

def findt(t):
    pm1t = pm1 - vm * t * (pm1 / NORM_M)
    zct = zfy - 3 * (t - 5.1) - 4.9 * 3.6 * 3.6
    pst = np.array([xct, yct, zct])
    xmt, _, _ = pm1t
    vectors1 = pm1t - s_p
    vectors2 = pm1t - bp
    vectorv1 = s_p - pst
    vectorv2 = bp - pst
```

```

norm_v1 = np.linalg.norm(vectors1)
d1 = np.linalg.norm(np.cross(vectors1, vectorv1)) / norm_v1
norm_v2 = np.linalg.norm(vectors2)
d2 = np.linalg.norm(np.cross(vectors2, vectorv2)) / norm_v2
# 求解直线s1, s2与球体的交点
t = sp.symbols('t')
a1, b1, c1 = vectors1
l1_x = s_p[0] + a1 * t
l1_y = s_p[1] + b1 * t
l1_z = s_p[2] + c1 * t
eq_in_t1 = (l1_x - xct)**2 + (l1_y - yct)**2 + (l1_z - zct)**2 -
    rs
t_sols1 = sp.solve(eq_in_t1, t)
inter_x1 = [float(l1_x.subs(t, sol)) for sol in t_sols1 if sol.
    is_real]
x1x = inter_x1[0] if len(inter_x1) > 0 else float('inf')
x2x = inter_x1[1] if len(inter_x1) > 1 else float('inf')
a2, b2, c2 = vectors2
l2_x = bp[0] + a2 * t
l2_y = bp[1] + b2 * t
l2_z = bp[2] + c2 * t
eq_in_t2 = (l2_x - xct)**2 + (l2_y - yct)**2 + (l2_z - zct)**2 -
    rs
t_sols2 = sp.solve(eq_in_t2, t)
inter_x2 = [float(l2_x.subs(t, sol)) for sol in t_sols2 if sol.
    is_real]
x3x = inter_x2[0] if len(inter_x2) > 0 else float('inf')
x4x = inter_x2[1] if len(inter_x2) > 1 else float('inf')
if d1 < 10 and d2 < 10 and (xmt > x1x or xmt > x2x or xmt > x3x
    or xmt > x4x):
    return True
return False

if __name__ == "__main__":
    start_time = time.time()
    t_start = 5.1
    t_end = 25.1
    # 分级搜索
    search_levels = [
        {'name': '1', 'dt': 0.1},
        {'name': '2', 'dt': 0.01},
        {'name': '3', 'dt': 0.001}
    ]
    search_zones = [(t_start, t_end)]
    for level in search_levels:
        print(f"[{level['name']}], 步长: {level['dt']}s")
        critical_zones = []
        for zones, zonee in search_zones:

```

```

        prev = findt(zones)
        for t in np.arange(zones + level['dt'], zonee + level['
            dt'], level['dt']):
            cur = findt(t)
            if cur != prev:
                critical_zones.append((t - level['dt'], t))
            prev = cur
        search_zones = critical_zones
        if not search_zones:
            break
    total_time = 0
    if search_zones:
        event_points = []
        for start, end in search_zones:
            if not findt(start) and findt(end):
                event_points.append({'type': 'start', 'time': end})
            elif findt(start) and not findt(end):
                event_points.append({'type': 'end', 'time': start})
        event_points.sort(key=lambda x: x['time'])
        shield_periods = []
        is_shielding = False
        start_t = 0
        for event in event_points:
            if event['type'] == 'start' and not is_shielding:
                start_t = event['time']
                is_shielding = True
            elif event['type'] == 'end' and is_shielding:
                shield_periods.append((start_t, event['time']))
                is_shielding = False
        for start_t, end_t in shield_periods:
            total_time += (end_t - start_t)
    end_time = time.time()
    print(f"在 t = {t_start} 到 t = {t_end} 秒的区间内,")
    print(f"有效遮蔽总时长为: {total_time:.3f} 秒。")

```

问题二代码

```

import math
import numpy as np
import sympy as sp
import time
from joblib import Parallel, delayed
import multiprocessing
from tqdm import tqdm
import matplotlib.pyplot as plt

G = 9.8
R_SMOKE_SQ = 100.0
T_EFFECTIVE = 20.0
P_M1_0 = np.array([20000.0, 0.0, 2000.0])
V_M1 = 300.0
P_FY1_0 = np.array([17800.0, 0.0, 1800.0])

x, y = sp.symbols('x y')
circle_eq = sp.Eq(x**2 + (y - 200)**2, 49)
line_eq = sp.Eq(y, -0.01 * x + 200)
solutions = sp.solve((circle_eq, line_eq), (x, y))
sol1, sol2 = solutions[0], solutions[1]
x1, y1 = float(sol1[0]), float(sol1[1])
x2, y2 = float(sol2[0]), float(sol2[1])
if x1 < x2:
    xs, ys = x1, y1
    xb, yb = x2, y2
else:
    xs, ys = x2, y2
    xb, yb = x1, y1
S_P = np.array([xs, ys, 7.0])
B_P = np.array([xb, yb, 0.0])

SEARCH_STAGES = [
    {'name': '第一阶段: 广域粗搜', 'grid_points': [6, 6, 5, 5]},
    {'name': '第二阶段: 区域精搜', 'grid_points': [10, 6, 5, 3]},
    {'name': '第三阶段: 焦点细搜', 'grid_points': [9, 5, 5, 4]},
]

NUM_CORES = max(1, multiprocessing.cpu_count() - 2)

def calculate_shield_time(individual):
    theta, v_fy, t_flight, t_fuse = individual
    t_symbol = sp.symbols('t_symbol')
    v_fy_vec = np.array([v_fy * math.cos(theta), v_fy * math.sin(
        theta), 0])
    p_drop = P_FY1_0 + v_fy_vec * t_flight
    t_detonation = t_flight + t_fuse

```

```

p_detonation = p_drop + v_fy_vec * t_fuse + np.array([0, 0, -0.5
    * G * t_fuse**2])

def _check_condition_at_t(t_val):
    pm_t = P_M1_0 - V_M1 * t_val * (P_M1_0 / np.linalg.norm(
        P_M1_0))
    zct = p_detonation[2] - 3.0 * (t_val - t_detonation)
    p_smoke_t = np.array([p_detonation[0], p_detonation[1], zct
        ])
    xmt, _, _ = pm_t; xct, yct, _ = p_smoke_t
    vectors1 = pm_t - S_P; vectors2 = pm_t - B_P
    vectorv1 = S_P - p_smoke_t; vectorv2 = B_P - p_smoke_t
    norm_v1 = np.linalg.norm(vectors1)
    d1 = np.linalg.norm(np.cross(vectors1, vectorv1)) / norm_v1
    norm_v2 = np.linalg.norm(vectors2)
    d2 = np.linalg.norm(np.cross(vectors2, vectorv2)) / norm_v2
    if not (d1 < 10 and d2 < 10):
        return False

    # 检查第一条直线的交点
    a1, b1, c1 = vectors1
    l1_x = S_P[0] + a1 * t_symbol; l1_y = S_P[1] + b1 * t_symbol
    ; l1_z = S_P[2] + c1 * t_symbol
    eq_in_t1 = (l1_x - xct)**2 + (l1_y - yct)**2 + (l1_z - zct)
        **2 - R_SMOKE_SQ
    t_sols1 = sp.solve(eq_in_t1, t_symbol)
    inter_x1 = [float(l1_x.subs(t_symbol, sol)) for sol in
        t_sols1 if sol.is_real]
    x1x = inter_x1[0] if len(inter_x1) > 0 else float('inf')
    x2x = inter_x1[1] if len(inter_x1) > 1 else float('inf')

    # 检查第二条直线的交点
    a2, b2, c2 = vectors2
    l2_x = B_P[0] + a2 * t_symbol; l2_y = B_P[1] + b2 * t_symbol
    ; l2_z = B_P[2] + c2 * t_symbol
    eq_in_t2 = (l2_x - xct)**2 + (l2_y - yct)**2 + (l2_z - zct)
        **2 - R_SMOKE_SQ
    t_sols2 = sp.solve(eq_in_t2, t_symbol)
    inter_x2 = [float(l2_x.subs(t_symbol, sol)) for sol in
        t_sols2 if sol.is_real]
    x3x = inter_x2[0] if len(inter_x2) > 0 else float('inf')
    x4x = inter_x2[1] if len(inter_x2) > 1 else float('inf')
    return d1 < 10 and d2 < 10 and (xmt > x1x or xmt > x2x or
        xmt > x3x or xmt > x4x)

t_start_win = t_detonation
t_end_win = t_detonation + T_EFFECTIVE
levels = [{'dt': 0.1}, {'dt': 0.01}, {'dt': 0.001}]

```

```

search_zones = [(t_start_win, t_end_win)]
for level in levels:
    critical_zones = []
    dt = level['dt']
    for zone_s, zone_e in search_zones:
        if zone_s >= zone_e:
            continue
        prev_state = _check_condition_at_t(zone_s)
        for t_cur in np.arange(zone_s + dt, zone_e + dt, dt):
            if t_cur > t_end_win:
                break
            cur_state = _check_condition_at_t(t_cur)
            if cur_state != prev_state:
                critical_zones.append((t_cur - dt, t_cur))
            prev_state = cur_state
        search_zones = critical_zones
    if not search_zones:
        return 0.0
total_time = 0.0
if search_zones:
    event_points = []
    for s, e in search_zones:
        if not _check_condition_at_t(s) and
            _check_condition_at_t(e):
            event_points.append({'type': 'start', 'time': e})
        elif _check_condition_at_t(s) and not
            _check_condition_at_t(e):
            event_points.append({'type': 'end', 'time': s})
    event_points.sort(key=lambda x: x['time'])
    is_shielding = False
    start_t = 0
    for event in event_points:
        if event['type'] == 'start' and not is_shielding:
            start_t = event['time']
            is_shielding = True
        elif event['type'] == 'end' and is_shielding:
            total_time += (event['time'] - start_t)
            is_shielding = False

    return total_time

def plot_search_process(all_stage_results):
    plt.rcParams['font.sans-serif'] = ['SimHei']
    plt.rcParams['axes.unicode_minus'] = False

    for i, stage_results in enumerate(all_stage_results):
        stage_name = SEARCH_STAGES[i]['name']
        fig, ax = plt.subplots(figsize=(12, 6))

```

```

x_axis = range(len(stage_results))
ax.plot(x_axis, stage_results, marker='.', linestyle='--',
        markersize=4, alpha=0.7)
ax.set_title(f'搜索过程分析 - {stage_name}', fontsize=16)
ax.set_xlabel('搜索次数', fontsize=12)
ax.set_ylabel('有效遮蔽时长 (秒)', fontsize=12)
ax.grid(True, linestyle='--', alpha=0.6)
ax.xaxis.set_major_locator(plt.MultipleLocator(100))
plt.tight_layout()
plt.savefig(f"{i+1}.png")
plt.show()

GENE_BOUNDS = [(0, math.pi/2), (131, 140), (0, 10), (0, 10)]
current_bounds = list(GENE_BOUNDS)
best_solution = None
best_score = -1
all_stage_results = []

for i, stage in enumerate(SEARCH_STAGES):
    stage_start_time = time.time()
    print(f"{stage['name']}")
    grid_points = stage['grid_points']
    total_points = np.prod(grid_points)
    dim_spaces = [np.linspace(b[0], b[1], num=p) for b, p in zip(
        current_bounds, grid_points)]
    param_combinations = list(np.array(np.meshgrid(*dim_spaces)).T.
        reshape(-1, 4))
    results = Parallel(n_jobs=NUM_CORES)(
        delayed(calculate_shield_time)(params) for params in tqdm(
            param_combinations, desc=f"计
            算 {stage['name']}"))
    all_stage_results.append(results)
    stage_best_score = max(results)
    if stage_best_score > best_score:
        best_score = stage_best_score
        best_idx = results.index(stage_best_score)
        best_solution = param_combinations[best_idx]
    if best_solution is None:
        best_solution = param_combinations[0]
    stage_end_time = time.time()
    print(f"本阶段最优得分: {stage_best_score:.4f} 秒")
    print(f"历史最优得分: {best_score:.4f} 秒")

    if i < len(SEARCH_STAGES) - 1:
        new_bounds = []
        step_sizes = [(b[1] - b[0]) / (p - 1 if p > 1 else 1) for b,
            p in zip(current_bounds, grid_points)]

```

```

        for dim in range(4):
            center = best_solution[dim]
            half_width = step_sizes[dim]
            lower_bound = max(center - half_width, GENE_BOUNDS[dim][0])
            upper_bound = min(center + half_width, GENE_BOUNDS[dim][1])
            new_bounds.append((lower_bound, upper_bound))
        current_bounds = new_bounds
total_end_time = time.time()

if best_solution is not None and best_score > 0:
    theta_opt, v_opt, t_flight_opt, t_fuse_opt = best_solution
    v_fy_vec_opt = np.array([v_opt * math.cos(theta_opt), v_opt *
        math.sin(theta_opt), 0])
    p_drop_opt = P_FY1_0 + v_fy_vec_opt * t_flight_opt
    p_detonation_opt = p_drop_opt + v_fy_vec_opt * t_fuse_opt + np.
        array([0, 0, -0.5 * G * t_fuse_opt**2])
    print(f"\n最优策略:")
    print(f"飞行方向 : {math.degrees(theta_opt):.4f} 度")
    print(f"飞行速度 : {v_opt:.4f} m/s")
    print(f"烟幕干扰弹投放点坐标 (X, Y, Z): ({p_drop_opt[0]:.2f}, {
        p_drop_opt[1]:.2f}, {p_drop_opt[2]:.2f})")
    print(f"烟幕干扰弹起爆点坐标 (X, Y, Z): ({p_detonation_opt[0]:.2f},
        {p_detonation_opt[1]:.2f}, {p_detonation_opt[2]:.2f})")
    print(f"\n最长遮蔽时间: {best_score:.4f} 秒")
plot_search_process(all_stage_results)

```

问题三代码

```
import math
import numpy as np
import time
from joblib import Parallel, delayed
import multiprocessing
from tqdm import tqdm
import matplotlib.pyplot as plt
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

G = 9.8
rs = 10.0
rs_sq = rs**2
t_eff = 20.0
pm0 = np.array([20000.0, 0.0, 2000.0])
vm = 300.0
pfy0 = np.array([17800.0, 0.0, 1800.0])
num_cores = max(1, multiprocessing.cpu_count() - 1)

xs, ys, xb, yb = -6.9965014875, 200.06996501, 6.9965014875,
199.93003499
sp = np.array([xs, ys, 7.0])
bp = np.array([xb, yb, 0.0])

def line_sphere_intersect(line_start, line_dir, sphere_center,
sphere_radius_sq):
    oc = line_start - sphere_center
    a = np.dot(line_dir, line_dir)
    b = 2 * np.dot(oc, line_dir)
    c = np.dot(oc, oc) - sphere_radius_sq
    disc = b*b - 4*a*c
    if disc < 0: return []
    sqrt_d = math.sqrt(disc)
    t1 = (-b - sqrt_d) / (2 * a); t2 = (-b + sqrt_d) / (2 * a)
    p1 = line_start + t1 * line_dir; p2 = line_start + t2 * line_dir
    return [p1, p2]

def calc_shield_time(ind, return_ind_times=False):
    theta, v_fy, t_flight1, t_fuse1, t_flight2, t_fuse2, t_flight3,
    t_fuse3 = ind
    if t_flight2 < 1.0 or t_flight3 < 1.0:
        return (0.0, [0.0, 0.0, 0.0]) if return_ind_times else 0.0
    v_fy_vec = np.array([v_fy * math.cos(theta), v_fy * math.sin(
theta), 0])
    smoke_events = []
    # 第1枚
```

```

t_drop1 = t_flight1
p_drop1 = pfy0 + v_fy_vec * t_drop1
t_det1 = t_drop1 + t_fuse1
p_det1 = p_drop1 + v_fy_vec * t_fuse1 + np.array([0, 0, -0.5 * G
    * t_fuse1**2])
# 第2枚
t_drop2 = t_drop1 + t_flight2
p_drop2 = pfy0 + v_fy_vec * t_drop2
t_det2 = t_drop2 + t_fuse2
p_det2 = p_drop2 + v_fy_vec * t_fuse2 + np.array([0, 0, -0.5 * G
    * t_fuse2**2])
# 第3枚
t_drop3 = t_drop2 + t_flight3
p_drop3 = pfy0 + v_fy_vec * t_drop3
t_det3 = t_drop3 + t_fuse3
p_det3 = p_drop3 + v_fy_vec * t_fuse3 + np.array([0, 0, -0.5 * G
    * t_fuse3**2])
smoke_events = [
    {'det_t': t_det1, 'det_p': p_det1},
    {'det_t': t_det2, 'det_p': p_det2},
    {'det_t': t_det3, 'det_p': p_det3}
]
# 约束检查
for event in smoke_events:
    if not (-10 <= event['det_p'][1] <= 210):
        return (0.0, [0.0, 0.0, 0.0]) if return_ind_times else
            0.0
    t_det = event['det_t']
    p_smoke_det = event['det_p']
    pm_at_det = pm0 - vm * t_det * (pm0 / np.linalg.norm(pm0))
    if p_smoke_det[0] >= pm_at_det[0]:
        return (0.0, [0.0, 0.0, 0.0]) if return_ind_times else
            0.0
# 遮蔽时长计算
dt_step = 0.05
min_det_t = min(e['det_t'] for e in smoke_events)
max_end_t = max(e['det_t'] + t_eff for e in smoke_events)
shielded_pts = set()
ind_shielded_pts = [set() for _ in range(3)]

for t_step in np.arange(min_det_t, max_end_t, dt_step):
    pm_t = pm0 - vm * t_step * (pm0 / np.linalg.norm(pm0))
    xmt, _, _ = pm_t
    any_shield = False
    active_smokes = []
    for i, event in enumerate(smoke_events):
        if event['det_t'] <= t_step <= event['det_t'] + t_eff:
            p_smoke_t = event['det_p'] - np.array([0, 0, 3.0 * (

```

```

        t_step - event['det_t']]))
        active_smokes.append({'id': i, 'pos': p_smoke_t})
if not active_smokes: continue
for smoke in active_smokes:
    i = smoke['id']; p_smoke_t = smoke['pos']
    vec1 = pm_t - sp; vec2 = pm_t - bp
    norm_v1 = np.linalg.norm(vec1); d1 = np.linalg.norm(np.
        cross(vec1, sp - p_smoke_t)) / norm_v1
    norm_v2 = np.linalg.norm(vec2); d2 = np.linalg.norm(np.
        cross(vec2, bp - p_smoke_t)) / norm_v2
    if not (d1 < rs and d2 < rs): continue
    inter1 = line_sphere_intersect(sp, vec1, p_smoke_t,
        rs_sq)
    inter2 = line_sphere_intersect(bp, vec2, p_smoke_t,
        rs_sq)
    x1x = inter1[0][0] if len(inter1) > 0 else float('inf')
    x2x = inter1[1][0] if len(inter1) > 1 else float('inf')
    x3x = inter2[0][0] if len(inter2) > 0 else float('inf')
    x4x = inter2[1][0] if len(inter2) > 1 else float('inf')
    if (xmt > x1x or xmt > x2x or xmt > x3x or xmt > x4x):
        any_shield = True
        ind_shielded_pts[i].add(int(t_step / dt_step))
if any_shield:
    shielded_pts.add(int(t_step / dt_step))

total_time = len(shielded_pts) * dt_step
if return_ind_times:
    ind_times = [len(pts) * dt_step for pts in ind_shielded_pts]
    return (total_time, ind_times)
else:
    return total_time

class DEOptimizer:
    def __init__(self, obj_func, bounds, pop_size=50, max_gen=100, F
        =0.8, CR=0.9):
        self.obj_func = obj_func; self.bounds = np.array(bounds);
        self.pop_size = pop_size
        self.max_gen = max_gen; self.F = F; self.CR = CR; self.dim =
            len(bounds)
        self.pop = np.zeros((pop_size, self.dim))
        for i in range(pop_size):
            for j in range(self.dim): self.pop[i, j] = np.random.
                uniform(self.bounds[j, 0], self.bounds[j, 1])
        results = Parallel(n_jobs=num_cores)(delayed(self.obj_func)(
            ind, True) for ind in tqdm(self.pop, desc="初始
            化"))
        self.fitness = np.array([res[0] for res in results])
        best_idx = np.argmax(self.fitness)

```

```

        self.best_fit = self.fitness[best_idx]
        self.best_sol = self.pop[best_idx].copy()
        self.best_ind_times = results[best_idx][1]
        self.conv_curve = [self.best_fit]

    def optimize(self):
        for gen in range(self.max_gen):
            trial_pop = np.zeros_like(self.pop)
            for i in range(self.pop_size):
                idxs = [idx for idx in range(self.pop_size) if idx
                        != i]; a, b, c = self.pop[np.random.choice(idxs,
                        3, replace=False)]
                mutant = np.clip(a + self.F * (b - c), self.bounds
                               [:, 0], self.bounds[:, 1])
                cross_pts = np.random.rand(self.dim) < self.CR
                if not np.any(cross_pts): cross_pts[np.random.
                    randint(0, self.dim)] = True
                trial = np.where(cross_pts, mutant, self.pop[i]);
                trial_pop[i] = trial

            trial_results = Parallel(n_jobs=num_cores)(delayed(self.
                obj_func)(ind, True) for ind in trial_pop)
            trial_fit = np.array([res[0] for res in trial_results])

            improve_mask = trial_fit >= self.fitness
            self.pop[improve_mask] = trial_pop[improve_mask]
            self.fitness[improve_mask] = trial_fit[improve_mask]

            curr_best_idx = np.argmax(self.fitness)
            if self.fitness[curr_best_idx] > self.best_fit:
                self.best_fit = self.fitness[curr_best_idx]
                self.best_sol = self.pop[curr_best_idx].copy()
                self.best_ind_times = trial_results[curr_best_idx
                    ][1]

            self.conv_curve.append(self.best_fit)
            print(f"DE 迭代 {gen + 1}/{self.max_gen}, 当前最
                优: {self.best_fit:.4f}s")
            return self.best_sol, self.best_fit, self.conv_curve, self.
                best_ind_times

bounds = [
    (0, 2 * math.pi), (70, 140),
    (0, 40), (0, 20),
    (1.0, 30), (0, 20),
    (1.0, 30), (0, 20),
]

```

```

de_opt = DEOptimizer(
    obj_func=calc_shield_time,
    bounds=bounds, pop_size=100, max_gen=300, F=0.8, CR=0.9
)
final_sol, final_score, history, final_ind_times = de_opt.optimize()

if final_score > 0:
    print(f"\n最优总遮蔽时间: {final_score:.4f} 秒")
    theta_opt, v_opt, t_flight1, t_fuse1, t_flight2, t_fuse2,
        t_flight3, t_fuse3 = final_sol
    v_fy_vec = np.array([v_opt * math.cos(theta_opt), v_opt * math.
        sin(theta_opt), 0])
    print(f" 飞行方向 : {math.degrees(theta_opt):.2f}度")
    print(f" 飞行速度 : {v_opt:.2f} m/s")
    # 计算每一枚的详细信息
    t_drop1 = t_flight1
    p_drop1 = pfy0 + v_fy_vec * t_drop1
    p_det1 = p_drop1 + v_fy_vec * t_fuse1 + np.array([0, 0, -0.5 * G
        * t_fuse1**2])
    t_drop2 = t_drop1 + t_flight2
    p_drop2 = pfy0 + v_fy_vec * t_drop2
    p_det2 = p_drop2 + v_fy_vec * t_fuse2 + np.array([0, 0, -0.5 * G
        * t_fuse2**2])
    t_drop3 = t_drop2 + t_flight3
    p_drop3 = pfy0 + v_fy_vec * t_drop3
    p_det3 = p_drop3 + v_fy_vec * t_fuse3 + np.array([0, 0, -0.5 * G
        * t_fuse3**2])
    all_drops = [
        {'t_flight': t_flight1, 't_fuse': t_fuse1, 'p_drop': p_drop1,
        'p_det': p_det1, 'type': '投放时
        间'},
        {'t_flight': t_flight2, 't_fuse': t_fuse2, 'p_drop': p_drop2,
        'p_det': p_det2, 'type': '投放间
        隔'},
        {'t_flight': t_flight3, 't_fuse': t_fuse3, 'p_drop': p_drop3,
        'p_det': p_det3, 'type': '投放间
        隔'}
    ]

    for i in range(3):
        event = all_drops[i]
        print(f"\n第 i+1 (*@枚烟幕弹")
        print(f"{event['type']}: {event['t_flight']:.2f}s")
        print(f"引信时长: {event['t_fuse']:.2f}s")
        print(f"投放点坐标 (X,Y,Z): ({event['p_drop'][0]:.2f}, {event
            ['p_drop'][1]:.2f}, {event['p_drop'][2]:.2f})")
        print(f"起爆点坐标 (X,Y,Z): ({event['p_det'][0]:.2f}, {event['
            p_det'][1]:.2f}, {event['p_det'][2]:.2f})")

```

```

for i, duration in enumerate(final_ind_times):
    print(f" 第 i+1 (*@枚烟幕弹: {duration:.4f} 秒")
plt.figure(figsize=(10, 6))
plt.plot(history)
plt.title('差分进化优化收敛曲线')
plt.xlabel('迭代次数')
plt.ylabel('最优有效遮蔽总时长 (秒)')
plt.grid(True)
max_score = max(history)
max_idx = history.index(max_score)
plt.scatter(max_idx, max_score,
            color='red', s=100, zorder=5, marker='*',
            label=f'最大值: {max_score:.4f}s')
plt.annotate(f'({max_idx}, {max_score:.4f})',
            xy=(max_idx, max_score),
            xytext=(max_idx + 5, max_score - 0.5),
            arrowprops=dict(facecolor='black', shrink=0.05,
                            width=1, headwidth=5))

plt.legend()
plt.savefig("t3.png")
plt.show()

```

问题四代码

```
import math
import numpy as np
import pandas as pd
import time
from joblib import Parallel, delayed
import multiprocessing
from tqdm import tqdm
import matplotlib.pyplot as plt

plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False
G = 9.8
rs = 10.0
rs_sq = rs**2
t_eff = 20.0
vm = 300.0
pm0 = np.array([20000.0, 0.0, 2000.0])
pfy0 = np.array([17800.0, 0.0, 1800.0])
pfy1 = np.array([12000.0, 1400.0, 1400.0])
pfy2 = np.array([6000.0, -3000.0, 700.0])
drone_pos = [pfy0, pfy1, pfy2]
sp = np.array([-6.9965, 200.0699, 7.0])
bp = np.array([6.9965, 199.9300, 0.0])
num_cores = max(1, multiprocessing.cpu_count() - 1)

def line_sphere_intersect(line_start, line_dir, sphere_center,
    sphere_radius_sq):
    oc = line_start - sphere_center; a = np.dot(line_dir, line_dir)
    b = 2 * np.dot(oc, line_dir); c = np.dot(oc, oc) -
        sphere_radius_sq
    disc = b*b - 4*a*c
    if disc < 0: return []
    sqrt_d = math.sqrt(disc)
    t1 = (-b - sqrt_d) / (2 * a); t2 = (-b + sqrt_d) / (2 * a)
    p1 = line_start + t1 * line_dir; p2 = line_start + t2 * line_dir
    return [p1, p2]

def calc_shield_time_q4(ind, return_ind_times=False):
    params = np.array(ind).reshape(3, 4)
    valid_smoke_events = []
    valid_drone_indices = []
    for i in range(3):
        theta, v_fy, t_flight, t_fuse = params[i]
        p_fy_init = drone_pos[i]
        v_fy_vec = np.array([v_fy * math.cos(theta), v_fy * math.sin
            (theta), 0])
```

```

p_drop = p_fy_init + v_fy_vec * t_flight
t_det = t_flight + t_fuse
p_det = p_drop + v_fy_vec * t_fuse + np.array([0, 0, -0.5 *
    G * t_fuse**2])
y_valid = -10 <= p_det[1] <= 210
pm_at_det = pm0 - vm * t_det * (pm0 / np.linalg.norm(pm0))
x_valid = p_det[0] < pm_at_det[0]
if y_valid and x_valid:
    valid_smoke_events.append({'det_t': t_det, 'det_p':
        p_det})
    valid_drone_indices.append(i)
if not valid_smoke_events: return (0.0, [0.0] * 3) if
    return ind_times else 0.0
shielded_pts = set(); ind_shielded_pts = [set() for _ in range
    (3)]
dt_step = 0.2; min_det_t = min(e['det_t'] for e in
    valid_smoke_events)
max_end_t = max(e['det_t'] + t_eff for e in valid_smoke_events)
for t_step in np.arange(min_det_t, max_end_t, dt_step):
    pm_t = pm0 - vm * t_step * (pm0 / np.linalg.norm(pm0)); xmt
        = pm_t[0]
    any_shield = False
    for event_idx, event in enumerate(valid_smoke_events):
        orig_drone_idx = valid_drone_indices[event_idx]
        if event['det_t'] <= t_step <= event['det_t'] + t_eff:
            p_smoke_t = event['det_p'] - np.array([0, 0, 3.0 * (
                t_step - event['det_t'])])
            vec1 = pm_t - sp; vec2 = pm_t - bp
            norm_v1 = np.linalg.norm(vec1); d1 = np.linalg.norm(
                np.cross(vec1, sp - p_smoke_t)) / norm_v1
            norm_v2 = np.linalg.norm(vec2); d2 = np.linalg.norm(
                np.cross(vec2, bp - p_smoke_t)) / norm_v2
            if not (d1 < rs and d2 < rs): continue
            inter1 = line_sphere_intersect(sp, vec1, p_smoke_t,
                rs_sq)
            inter2 = line_sphere_intersect(bp, vec2, p_smoke_t,
                rs_sq)
            is_shielded = False
            if inter1 and any(intersect[0] < xmt for intersect
                in inter1): is_shielded = True
            if inter2 and any(intersect[0] < xmt for intersect
                in inter2): is_shielded = True
            if is_shielded:
                any_shield = True
                ind_shielded_pts[orig_drone_idx].add(int(t_step
                    / dt_step))
        if any_shield: shielded_pts.add(int(t_step / dt_step))
total_time = len(shielded_pts) * dt_step

```

```

ind_times = [len(pts) * dt_step for pts in ind_shielded_pts]
return (total_time, ind_times) if return_ind_times else
    total_time

```

```

class DEOptimizer:

```

```

    def __init__(self, obj_func, bounds, pop_size, max_gen, F, CR):
        self.obj_func = obj_func; self.bounds = np.array(bounds);
        self.pop_size = pop_size
        self.max_gen = max_gen; self.F = F; self.CR = CR; self.dim =
            len(bounds)
        self.pop = np.random.rand(pop_size, self.dim) * (self.bounds
           [:, 1] - self.bounds[:, 0]) + self.bounds[:, 0]
        results = Parallel(n_jobs=num_cores)(delayed(self.obj_func)(
            ind, True) for ind in tqdm(self.pop, desc="初始
            化"))

```

```

        self.fitness = np.array([res[0] for res in results])
        best_idx = np.argmax(self.fitness)
        self.best_fit = self.fitness[best_idx]
        self.best_sol = self.pop[best_idx].copy()
        _, self.best_ind_times = self.obj_func(self.best_sol, True)
        self.conv_curve = [self.best_fit]

```

```

    def optimize(self):

```

```

        for gen in range(self.max_gen):
            trial_pop = np.zeros_like(self.pop)
            for i in range(self.pop_size):
                idxs = [idx for idx in range(self.pop_size) if idx
                    != i]; a, b, c = self.pop[np.random.choice(idxs,
                        3, replace=False)]
                mutant = np.clip(a + self.F * (b - c), self.bounds
                   [:, 0], self.bounds[:, 1])
                cross_pts = np.random.rand(self.dim) < self.CR
                if not np.any(cross_pts): cross_pts[np.random.
                    randint(0, self.dim)] = True
                trial = np.where(cross_pts, mutant, self.pop[i]);
                trial_pop[i] = trial
            trial_results = Parallel(n_jobs=num_cores)(delayed(self.
                obj_func)(ind, True) for ind in trial_pop)
            trial_fit = np.array([res[0] for res in trial_results])
            improve_mask = trial_fit >= self.fitness
            self.pop[improve_mask] = trial_pop[improve_mask]
            self.fitness[improve_mask] = trial_fit[improve_mask]
            curr_best_idx = np.argmax(self.fitness)
            if self.fitness[curr_best_idx] > self.best_fit:
                self.best_fit = self.fitness[curr_best_idx]
                self.best_sol = self.pop[curr_best_idx].copy()
            _, self.best_ind_times = self.obj_func(self.best_sol,
                True)
            self.conv_curve.append(self.best_fit)

```

```

        eff_drones = [f"FY{i+1}" for i, duration in enumerate(
            self.best_ind_times) if duration > 0]
        eff_drones_str = ", ".join(eff_drones) if eff_drones
        else "无"
        print(f"DE 迭代 {gen + 1}/{self.max_gen}, 最优遮蔽总时
            长: {self.best_fit:.4f}s, 起作用无人
            机: {eff_drones_str}")

    return self.best_sol, self.best_fit, self.conv_curve

fy3_anchor_sol = np.array([1.53746, 104.13, 15.56, 4.05])
fy2_anchor_sol = np.array([4.79, 86.37, 11.18, 4.82])
search_half_widths = {'theta': math.radians(15), 'v_fy': 10.0, '
    t_flight': 8.0, 't_fuse': 5.0}
GENE_BOUNDS_FY2 = [(math.radians(200), math.radians(340)), (70, 140),
    (0, 80), (0, 25)]
bounds_fy2_focused = []
for i in range(4):
    center, half_width = fy2_anchor_sol[i], list(search_half_widths.
        values())[i]
    lower, upper = center - half_width, center + half_width
    lower = max(lower, GENE_BOUNDS_FY2[i][0]); upper = min(upper,
        GENE_BOUNDS_FY2[i][1])
    bounds_fy2_focused.append((lower, upper))

GENE_BOUNDS_FY3 = [(math.radians(20), math.radians(160)), (70, 140),
    (20, 80), (0, 25)]
bounds_fy3_focused = []
for i in range(4):
    center, half_width = fy3_anchor_sol[i], list(search_half_widths.
        values())[i]
    lower, upper = center - half_width, center + half_width
    lower = max(lower, GENE_BOUNDS_FY3[i][0]); upper = min(upper,
        GENE_BOUNDS_FY3[i][1])
    bounds_fy3_focused.append((lower, upper))

# --- 5. 组合所有无人机的最终边界 ---
bounds_fy1 = [(0, 2 * math.pi), (70, 140), (0, 80), (0, 25)]
bounds = bounds_fy1 + bounds_fy2_focused + bounds_fy3_focused
print("FY2的搜索边界已根据锚点解进行收缩:")
print(f" 角度(rad): ({bounds_fy2_focused[0][0]:.2f}, {
    bounds_fy2_focused[0][1]:.2f})")
print(f" 速度(m/s): ({bounds_fy2_focused[1][0]:.2f}, {
    bounds_fy2_focused[1][1]:.2f})")
print("FY3的搜索边界已根据锚点解进行收缩:")
print(f" 角度(rad): ({bounds_fy3_focused[0][0]:.2f}, {
    bounds_fy3_focused[0][1]:.2f})")
print(f" 速度(m/s): ({bounds_fy3_focused[1][0]:.2f}, {
    bounds_fy3_focused[1][1]:.2f})")

```

```

main_optimizer = DEOptimizer(
    obj_func=calc_shield_time_q4, bounds=bounds,
    pop_size=200, max_gen=500, F=0.8, CR=0.9
)
final_sol, final_score, history = main_optimizer.optimize()

_, final_ind_times = calc_shield_time_q4(final_sol, True)
print(f"\n找到的最优策略对应的最长总遮蔽时间: {final_score:.4f} 秒")

params = np.array(final_sol).reshape(3, 4)
results_data = []
for i in range(3):
    theta, v_fy, t_flight, t_fuse = params[i]
    p_fy_init = drone_pos[i]
    v_fy_vec = np.array([v_fy * math.cos(theta), v_fy * math.sin(
        theta), 0])
    p_drop = p_fy_init + v_fy_vec * t_flight
    t_det = t_flight + t_fuse
    p_det_coord = p_drop + v_fy_vec * t_fuse + np.array([0, 0, -0.5
        * G * t_fuse**2])
    drone_name = f"FY{i+1}"
    print(f"\n无人机 {drone_name} 的策略")
    print(f" 飞行方向: {math.degrees(theta):.2f} 度, 速
        度: {v_fy:.2f} m/s, 飞行时长: {t_flight:.2f} s")
    print(f" 引信时长: {t_fuse:.2f} s, 起爆时刻: {t_det:.2f} s")
    print(f" 投放点 (X,Y,Z): ({p_drop[0]:.2f}, {p_drop[1]:.2f}, {
        p_drop[2]:.2f})")
    print(f" 起爆点 (X,Y,Z): ({p_det_coord[0]:.2f}, {p_det_coord
        [1]:.2f}, {p_det_coord[2]:.2f})")
    results_data.append({
        '无人机编号': drone_name, '飞行方向': math.degrees(theta), '飞行
        速度': v_fy,
        '投放点X': p_drop[0], '投放点Y': p_drop[1], '投放
        点Z': p_drop[2],
        '起爆点X': p_det_coord[0], '起爆点Y': p_det_coord[1], '起爆
        点Z': p_det_coord[2],
    })

for i, duration in enumerate(final_ind_times):
    print(f" 第 {i+1} 枚烟幕弹 (由FY{i+1}投放): {duration:.4f} 秒")

df_results = pd.DataFrame(results_data)
df_results.to_excel("result4.xlsx", index=False, float_format="%.4f"
)

plt.figure(figsize=(12, 7))
plt.plot(history, label='每代最优值', linewidth=1.5, alpha=0.8)

```

```

plt.title('差分进化优化收敛曲线', fontsize=16)
plt.xlabel('迭代次数', fontsize=12)
plt.ylabel('最长有效遮蔽总时长 (秒)', fontsize=12)
plt.grid(True, linestyle='--', alpha=0.6)

if history:
    max_score = max(history)
    max_gen_idx = next(i for i, v in enumerate(history) if v ==
                        max_score)
    plt.scatter(max_gen_idx, max_score,
                color='red',
                s=120,
                marker='*',
                zorder=5,
                label=f'历史最大值: {max_score:.4f}s')
    annotation_text = f'第 {max_gen_idx} 代\n最优值: {max_score:.4f}s'
    plt.annotate(annotation_text,
                 xy=(max_gen_idx, max_score),
                 xytext=(max_gen_idx + 20, max_score - max(1,
                     max_score/10)),
                 arrowprops=dict(facecolor='black', shrink=0.05,
                     width=1, headwidth=6),
                 fontsize=11,
                 bbox=dict(boxstyle="round,pad=0.5", fc="yellow", ec
                     ="black", lw=1, alpha=0.7))

plt.legend(fontsize=12)
plt.tight_layout()
plt.savefig("t4.png")
plt.show()

```

问题五代码

```
import math
import numpy as np
import pandas as pd
import time
import multiprocessing
from tqdm import tqdm
import sympy as sp

vm = 300
G = 9.8
rs = 10
sink_speed = 3
t_eff = 20
rs_sq = rs**2
max_retries = 50
drones = {
    "FY1": {"init_pos": np.array([17800, 0, 1800]), "max_smoke": 3,
            "speed_range": [70, 140]},
    "FY2": {"init_pos": np.array([12000, 1400, 1400]), "max_smoke": 3,
            "speed_range": [70, 140]},
    "FY3": {"init_pos": np.array([6000, -3000, 700]), "max_smoke": 3,
            "speed_range": [70, 140]},
    "FY4": {"init_pos": np.array([11000, 2000, 1800]), "max_smoke": 3,
            "speed_range": [70, 140]},
    "FY5": {"init_pos": np.array([13000, -2000, 1300]), "max_smoke": 3,
            "speed_range": [70, 140]}
}
missiles = {
    "M1": {"init_pos": np.array([20000, 0, 2000])},
    "M2": {"init_pos": np.array([19000, 600, 2100])},
    "M3": {"init_pos": np.array([18000, -600, 1900])}
}
y_range = [-10, 210]

x, y = sp.symbols('x y')
circle_eq = sp.Eq(x**2 + (y - 200)**2, 49)
line_eq = sp.Eq(y, -0.01 * x + 200)
solutions = sp.solve((circle_eq, line_eq), (x, y))
x1, y1 = float(solutions[0][0]), float(solutions[0][1])
x2, y2 = float(solutions[1][0]), float(solutions[1][1])
sp = np.array([min(x1, x2), solutions[0][1] if min(x1, x2) == x1
               else solutions[1][1], 7.0])
bp = np.array([max(x1, x2), solutions[0][1] if max(x1, x2) == x1
               else solutions[1][1], 0.0])

def get_missile_pos(missile_id, t):
```

```

    p0 = missiles[missile_id]["init_pos"]
    direction = -p0 / np.linalg.norm(p0)
    return p0 + direction * vm * t

def line_sphere_intersect(p1, dir_vec, sphere_center,
    sphere_radius_sq):
    oc = p1 - sphere_center
    a = np.dot(dir_vec, dir_vec)
    b = 2 * np.dot(oc, dir_vec)
    c = np.dot(oc, oc) - sphere_radius_sq
    disc = b**2 - 4*a*c
    return disc >= 0

def calc_shield_time(params, drone_id, missile_id):
    theta, v_fy, t_flight, t_fuse = params
    p_fy_init = drones[drone_id]["init_pos"]
    v_fy_vec = np.array([v_fy * math.cos(theta), v_fy * math.sin(
        theta), 0])
    p_drop = p_fy_init + v_fy_vec * t_flight
    t_det = t_flight + t_fuse
    p_det = p_drop + v_fy_vec * t_fuse + np.array([0, 0, -0.5 * G *
        t_fuse**2])

    pm_at_det = get_missile_pos(missile_id, t_det)
    if not (y_range[0] <= p_det[1] <= y_range[1]): return 0.0
    if p_det[0] >= pm_at_det[0]: return 0.0

    total_time = 0
    time_pts = np.arange(t_det, t_det + t_eff, 0.1)
    for t in time_pts:
        pm_t = get_missile_pos(missile_id, t)
        p_smoke_t = p_det - np.array([0, 0, sink_speed * (t - t_det)
            ])
        shield_s = line_sphere_intersect(sp, pm_t - sp, p_smoke_t,
            rs_sq)
        shield_b = line_sphere_intersect(bp, pm_t - bp, p_smoke_t,
            rs_sq)
        if shield_s and shield_b: total_time += 0.1
    return total_time

def get_known_solutions():
    known_solutions = {
        "FY1": {
            "theta": math.radians(179.64),
            "v_fy": 139.76,
            "t_flight": 2.0,
            "t_fuse": 1.0
        },

```

```

        "FY2": {
            "theta": math.radians(241.7),
            "v_fy": 132.2,
            "t_flight": 20.0,
            "t_fuse": 10.0
        },
        "FY3": {
            "theta": math.radians(112.2),
            "v_fy": 135.7,
            "t_flight": 20.0,
            "t_fuse": 10.0
        },
        "FY4": {
            "theta": math.radians(253.5),
            "v_fy": 138.5,
            "t_flight": 20.0,
            "t_fuse": 10.0
        },
        "FY5": {
            "theta": math.radians(104.4),
            "v_fy": 135.48,
            "t_flight": 20.0,
            "t_fuse": 10.0
        }
    }
}
return known_solutions

class LocalSearchOptimizer:
    def __init__(self, bounds, max_iter=200, step_size=0.1):
        self.bounds = bounds
        self.max_iter = max_iter
        self.step_size = step_size
        self.dim = len(bounds)

    def optimize(self, fit_func, init_sol):
        curr_sol = np.array(init_sol)
        curr_fit = fit_func(curr_sol)
        best_sol = curr_sol.copy()
        best_fit = curr_fit
        step_sizes = np.array([self.step_size] * self.dim)
        no_improve_count = 0

        print(f"初始种子适应度: {curr_fit:.4f}s")
        for iter in range(self.max_iter):
            improved = False
            for dim in range(self.dim):
                for direction in [-1, 1]:
                    new_sol = curr_sol.copy()

```

```

        new_sol[dim] += direction * step_sizes[dim]
        new_sol[dim] = np.clip(new_sol[dim], self.bounds
                                [dim][0], self.bounds[dim][1])
        new_fit = fit_func(new_sol)
        if new_fit > best_fit:
            best_sol = new_sol.copy()
            best_fit = new_fit
            curr_sol = new_sol.copy()
            curr_fit = new_fit
            step_sizes[dim] *= 1.1
            improved = True
            break
        else:
            step_sizes[dim] *= 0.9
    if not improved:
        no_improve_count += 1
        if no_improve_count > 25:
            print(f" 局部优化在 {iter + 1} 代后收敛。")
            break
    else:
        no_improve_count = 0
    print(f" 局部优化后最佳适应度: {best_fit:.4f}s")
    return best_sol, best_fit

def save_results(final_strategies, filename="t5.xlsx"):
    data = []
    for drone_id, result in final_strategies.items():
        theta, v_fy, t_drop, t_fuse = result["params"]
        data.append({
            "无人机": drone_id, "(*@目标导弹": result["missile_id"],
            "飞行角度(度)": math.degrees(theta), "飞行速度(m/s)": v_fy,
            "投放时间(s)": t_drop, "引信时间(s)": t_fuse,
            "最终遮蔽时间(s)": result["shield_time"]
        })
    df = pd.DataFrame(data)
    df.to_excel(filename, index=False)
    print(f"\n结果已保存到 {filename}")
    return df

multiprocessing.freeze_support()
start_time = time.time()
assignments = {"FY1": "M1", "FY2": "M2", "FY3": "M3", "FY4": "M2", "
FY5": "M3"}
known_solutions = get_known_solutions()
final_strategies = {}

for drone_id, missile_id in assignments.items():
    print(f"\n--- 正在优化 {drone_id} -> {missile_id} ---")

```

```

known_params = known_solutions[drone_id]
init_sol = [
    known_params["theta"],
    known_params["v_fy"],
    known_params["t_flight"],
    known_params["t_fuse"]
]
init_shield_time = calc_shield_time(init_sol, drone_id,
    missile_id)
print(f" 已知参数的初始遮蔽时间: {init_shield_time:.4f}s")
drone_pos = drones[drone_id]["init_pos"]
theta_range = (known_params["theta"] - 0.01, known_params["theta"]
    + 0.01)
v_fy_range = (known_params["v_fy"] - 0.01, known_params["v_fy"]
    + 0.01)
bounds = [theta_range, v_fy_range, (10, 80), (0, 25)]
fit_func = lambda params: calc_shield_time(params, drone_id,
    missile_id)
local_optimizer = LocalSearchOptimizer(bounds=bounds)
best_params, best_shield_time = local_optimizer.optimize(
    fit_func=fit_func,
    init_sol=init_sol
)
if best_shield_time > 0:
    final_strategies[drone_id] = {
        "missile_id": missile_id,
        "params": best_params,
        "shield_time": best_shield_time
    }
if final_strategies:
    result_df = save_results(final_strategies)
    print("\n最终方案详情:")
    print(result_df.to_string(index=False))

    total_shield_time = sum(res["shield_time"] for res in
        final_strategies.values())
    print(f"\n总计有效遮蔽时间: {total_shield_time:.2f} 秒")
else:
    print("未找到任何有效的遮蔽方案。")

```